CREATION OF AN INNOVATIVE PLATFORM FOR AUTOMATION OF PURCHASE AND MANAGEMENT OF CONTRACTS FOR THE SUPPLY OF GOODS

By

Sergaziyev Nurkhat, Talipov Zhenis, Aitkozha Arshat

Department of Computer Engineering
Astana IT University

SE-2204, SE-2219 — Software Engineering
Supervisor: Makatov A.K.

Astana, 2025

Table of Contents

Abstracts

Abstract

This diploma project introduces the design and implementation of Zhan.Store — a full-cycle digital platform for automating B2B procurement processes in Kazakhstan. Targeted at small and medium-sized enterprises (SMEs), the system enables users to browse a product catalog, manage supplier-segmented shopping carts, generate legally compliant contracts, apply digital signatures, and process payments through integrated local services such as Fondy.

The frontend is built with Next.js using the App Router, TypeScript, and Tailwind CSS, while the backend leverages a modular monolithic architecture developed in Go (Golang). Additionally, a mobile application was developed using Flutter, following the MVVM pattern and BLoC for state management, ensuring a smooth and consistent user experience across platforms.

The platform also supports multi-supplier orders, document rendering for contracts and delivery acts, JWT-based authentication, and a future-ready e-signature integration. By addressing the lack of localized procurement tools for SMEs, Zhan.Store contributes to the digitalization of commerce in Kazakhstan, offering a secure, scalable, and user-friendly solution aligned with local business and legal standards.

Аннотация

Бұл дипломдық жобада Қазақстандағы шағын және орта бизнеске (ШОБ) арналған B2B сатып алуларын автоматтандыруға бағытталған Zhan.Store цифрлық платформасы жасалып, жүзеге асырылды. Жүйе қолданушыларға өнім каталогын шолуға, жеткізушілер бойынша бөлінген себетті басқаруға, заңды талаптарға сәйкес келісімшарттарды автоматты түрде жасауға, оларды цифрлық түрде қол қоюға және жергілікті төлем жүйелері арқылы (мысалы, Fondy) төлем жасауға мүмкіндік береді.

Фронтенд Next.js (App Router), TypeScript және Tailwind CSS арқылы жасалған. Бэкенд Go (Golang) тілінде модульді монолитті архитектура негізінде құрылды. Со-

нымен қатар Flutter арқылы MVVM үлгісі мен BLoC күйін басқару әдісі қолданылған мобильді қосымша жасалды.

Платформа көпжеткізушілік тапсырыстарды, келісімшарттар мен жеткізу актілерін көруді, JWT негізіндегі аутентификацияны және болашақта Қазақстанның электрондық қолтаңба инфрақұрылымын біріктіруді қолдайды. Zhan.Store — отандық бизнеске қолжетімді, қауіпсіз және заңмен үйлесімді шешім ұсына отырып, елдегі цифрлық сауданың дамуына өз үлесін қосатын инновациялық жоба.

Аннотация

Данный дипломный проект посвящен разработке и внедрению цифровой платформы Zhan.Store — решения для полной автоматизации B2B-закупок, ориентированного на малый и средний бизнес Казахстана. Платформа позволяет пользователям просматривать каталог товаров, управлять корзиной, разделенной по поставщикам, автоматически формировать контракты в соответствии с юридическими требованиями, подписывать их в цифровом виде и осуществлять оплату через локальные сервисы, такие как Fondy.

Фронтенд реализован на Next.js (App Router) с использованием TypeScript и Tailwind CSS. Бэкенд построен на языке Go в виде модульного монолита, что обеспечивает удобную масштабируемость и независимость доменных контекстов. Также разработано мобильное приложение на Flutter, использующее архитектуру MVVM и управление состоянием через BLoC.

Платформа поддерживает многопоставочные заказы, отображение контрактов и актов, аутентификацию через JWT и в будущем планирует внедрение электронной подписи через национальные сервисы. Zhan.Store закрывает пробел в цифровых решениях для МСБ в Казахстане и способствует развитию цифровой экономики страны.

Definitions

- **Contract**: A formal and legally binding agreement between a supplier and a customer outlining the terms of delivery, pricing, and responsibilities.
- **Authentication Token**: A secure, digitally signed data object used to verify the identity of a user in communication with backend systems.
- **Digital Signature**: A cryptographic technique used to validate the authenticity and integrity of a digital message or document.
- **Frontend Interface**: The part of the system that users interact with directly through a web browser or mobile application.
- **Backend System**: The server-side component responsible for processing business logic, data storage, and inter-module operations.
- **Middleware**: Software components that intercept and process data between different layers or modules, typically used for authentication, logging, and error handling.
- **PostgreSQL Database**: An open-source, object-relational database system used for storing and managing structured data.
- **Containerization**: The practice of packaging software and its dependencies into isolated units called containers to ensure consistency across different computing environments.
- **REST API**: A type of web service that allows communication between systems over HTTP using resource-based URLs and standard operations such as GET, POST, PUT, and DELETE.
- **Data Transfer Object**: A data structure used to transfer data between software application layers, typically without any business logic.

Designations and abbreviations

Following designations and abbreviations are used in this work:

API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
NCA	National Certification Authority
DS	Digital Signature
CSS	Cascading Style Sheets
EDS	Electronic Digital Signature
XML	eXtensible Markup Language
IIN	Individual Identification Number
MVVM	Model-View-ViewModel
CI/CD	Continuous Integration and Continuous Delivery
DDD	Domain-Driven Design
ERD	Entity-Relationship Diagram
RBAC	Role-Based Access Control
ORM	Object-Relational Mapping
CDN	Content Delivery Network
SQL	Structured Query Language
BLoC	Business Logic Component
DTO	Data Transfer Object
CRUD	Create, Read, Update, Delete
TLS	Transport Layer Security
PDF	Portable Document Format
UI	User Interface
UX	User Experience

Introduction

In today's rapidly evolving business environment, automation has become not just a competitive advantage but a necessity for operational efficiency. Procurement and contract management—two foundational components of any organization's supply chain—are often handled using outdated, manual methods, particularly in small and medium-sized enterprises (SMEs). These processes are traditionally bogged down by paper-based approvals, fragmented communication with suppliers, human errors in pricing or documentation, and an overall lack of transparency in decision-making. This project presents the development of a digital procurement automation platform designed to streamline and digitize the full cycle of B2B purchasing and contract oversight for SMEs in Kazakhstan.

The primary goal of this work is to build a comprehensive system that automates the creation, approval, and management of procurement requests, facilitates intelligent supplier selection, handles contract generation and tracking, and supports real-time order status monitoring. Additionally, the platform enables digital handling of post-payment documentation, including acceptance certificates and, in the future, legally binding electronic signatures. Its architecture consists of three tightly integrated components: a mobile application developed using Flutter, a backend system powered by Golang and PostgreSQL, and a responsive web interface tailored for business users. These components communicate via secure RESTful APIs and are documented through Swagger for integration scalability.

The core problem the platform addresses is the inefficient and error-prone nature of traditional procurement in SMEs. In most cases, businesses rely on Excel spreadsheets, phone calls, and emails to search for suppliers, compare prices, and coordinate orders. This often results in disorganized communication, price misalignment, slow approvals, and the absence of clear contract oversight. Furthermore, businesses suffer from a lack of procurement analytics, making it difficult to assess performance, track spending, or optimize supplier relationships. These issues are especially acute in Kazakhstan, where SMEs dominate the economic landscape but lack access to affordable and locally adapted procurement tools.

What sets this platform apart is its specific orientation toward local business realities. While international solutions exist, they often fail to account for Kazakhstan's tax

regulations, invoice standards, language preferences, and payment systems. As a result, the adoption rate of foreign software remains low. This platform is purpose-built for Kazakhstan's SME ecosystem, supporting local currency (tenge), regional logistics, national payment gateways such as Fondy, and government-compliant documentation formats.

The system automates the following key processes:

- Structured procurement request creation and approval
- AI-assisted supplier recommendation based on price, delivery terms, and reliability
- Dynamic product catalog with real-time filtering by supplier
- Intelligent multi-supplier shopping cart with separate logistics and pricing logic
- Transparent cost breakdowns including minimum order thresholds and delivery fees
- Automatic contract drafting upon order confirmation
- Digital confirmation of goods received with status tracking per supplier
- Post-order document flow including acceptance certificates signed through user dashboard
- Planned integration with digital signature infrastructure for legal validity

The intended users of the system are small and medium-sized businesses engaged in frequent procurement of goods—particularly in the retail, food service, distribution, and manufacturing sectors. The platform follows a B2B model, allowing business clients to register, browse a curated catalog of suppliers and products, and execute multi-supplier purchases in a seamless interface. Each transaction is split automatically per supplier to ensure that logistical and contractual obligations are clearly defined. Suppliers also benefit from a dedicated dashboard, where they can review incoming orders, update product availability, and track the progress of contract confirmations.

From a business value perspective, the platform helps SMEs:

- Reduce procurement time by eliminating manual coordination
- Minimize pricing and documentation errors
- Improve compliance with local contract and tax regulations

- Track procurement activity for analytics and reporting
- Increase transparency and traceability in business operations

The objectives of this project can be summarized as follows:

- Design a scalable, secure client-server architecture that supports web and mobile interfaces
- Implement user-friendly workflows for procurement, supplier interaction, and document management
- Integrate local payment and invoice systems for legal and financial compliance
- Build supplier-side interfaces for managing incoming orders and confirming fulfillment
- Enable customers to digitally approve order fulfillment with future-ready signature functionality
- Provide clear and intuitive analytics dashboards for procurement insights and historical tracking

By combining localized functionality with modern digital workflows, this platform addresses a clear gap in Kazakhstan's SME infrastructure. It empowers small businesses to operate with the same level of procurement control and efficiency as larger enterprises—without the complexity or cost of enterprise-grade systems. This relevance, combined with real-world user research and full-stack technical implementation, makes the project a meaningful contribution to both the software engineering field and the broader economic digitalization effort underway in Central Asia.

1 Literature Review

1.1 Overview of Procurement Automation

Procurement automation refers to the integration of digital technologies into procurement workflows—covering tasks such as requisitioning, supplier selection, order management, invoice processing, and contract tracking. In global enterprise environments, these systems have become essential tools that enhance efficiency, reduce manual workload, and improve transparency across the supply chain (Kuznetsov, 2024).

Automated procurement platforms typically provide features such as centralized supplier directories, price comparison engines, real-time inventory updates, automatic invoice matching, and digital contract archives. These capabilities enable faster decision-making, standardized documentation, and data-driven purchasing strategies. In volatile market environments, automation also supports better risk management and supplier diversification.

However, for small and medium-sized enterprises (SMEs), particularly in developing economies like Kazakhstan, access to such technologies remains limited. Most SMEs still rely on manual tools—Excel spreadsheets, phone calls, email chains, and paper contracts. This leads to delays, pricing inconsistencies, missed deadlines, and little visibility into procurement performance. With minimal IT infrastructure and administrative support, these inefficiencies are often accepted as the norm.

A key challenge is that international procurement solutions are typically designed for large enterprises. They are expensive, complex to implement, and lack localization—such as Kazakh language support, integration with local payment gateways, or compliance with national tax standards. As a result, SMEs in Kazakhstan are effectively excluded from the digital procurement revolution.

This technological divide contributes to operational inefficiencies and limits the scalability of Kazakhstan's SME sector. The Asian Development Bank (2020) highlights that the lack of accessible, localized infrastructure—particularly outside major cities—is a major constraint to SME growth. Without digital procurement tools, businesses face increased operational costs, lower competitiveness, and a higher risk of supply chain disruptions.

Meanwhile, the rapid rise of e-commerce in Kazakhstan illustrates that digital adoption

is possible when platforms are intuitive, affordable, and locally relevant. This trend reinforces the need for a dedicated procurement automation solution tailored to the realities of Kazakhstani SMEs—bridging the gap between manual processes and fully integrated supply chain management (Asian Development Bank, 2020; Kuznetsov, 2024).

1.2 Classical Procurement Models

Traditional procurement in SMEs typically follows a manual and sequential flow: identifying organizational needs, contacting multiple suppliers to solicit prices, negotiating terms through phone calls or emails, and managing orders and contracts using paper-based documentation. This linear and fragmented approach, while simple on the surface, introduces a number of hidden inefficiencies that compound over time.

Among the most common issues associated with this outdated model are:

- **Delayed decision-making and order fulfillment**: Without automated tools for supplier comparison, lead time estimation, or order tracking, procurement decisions are slow, and fulfillment timelines are often unclear or inaccurate.
- **Errors in calculations and invoicing**: Manual data entry into spreadsheets or handwritten records increases the likelihood of numerical mistakes, incorrect totals, and inconsistencies between purchase orders and supplier invoices.
- **Lack of transparency in supplier selection**: In the absence of a centralized database or performance tracking system, supplier choices are often made based on familiarity or convenience, rather than pricing, quality, or delivery history.
- **No procurement history or data analytics**: Without digital tracking, there is no structured archive of previous purchases, making it difficult to perform trend analysis, audit spending, or optimize supplier relationships.
- **Inconsistent documentation**: Paper contracts, printed invoices, and informal agreements are prone to being lost, misfiled, or unreadable—especially when managed by non-specialist staff or across multiple departments.

In micro-businesses with minimal procurement complexity, this approach may be manageable in the short term. However, as companies grow in scale or frequency of orders, the lack of formalization becomes a bottleneck. Miscommunications, order duplication, and undocumented agreements begin to introduce operational and financial risks.

Additionally, this manual system makes it nearly impossible to scale operations efficiently or maintain compliance with evolving financial and tax regulations. For businesses involved in public sector tenders or cross-border trade, the inability to produce standardized documentation and maintain audit trails can become a serious legal liability.

For SMEs in Kazakhstan, these challenges are particularly acute. According to Sidorov (2023), many small enterprises continue to rely on paper records and informal supplier relationships due to the *absence of affordable, localized, and easy-to-use procurement solutions*. The available tools are either too complex, built for large enterprises, or not adapted to the Kazakhstani legal and logistical context.

Moreover, in regional areas where digital literacy is lower and access to IT support is limited, the transition from manual to digital procurement requires not just software but also simplified workflows, training, and trust in digital processes. As a result, SMEs are locked into inefficient models—not because of resistance to innovation, but due to the lack of suitable infrastructure and tools.

This persistent reliance on outdated procurement practices prevents SMEs from taking advantage of bulk discounts, optimizing delivery cycles, or negotiating better terms based on historical performance. It also limits their ability to participate in national and regional supply chains, particularly as larger enterprises increasingly demand digital integration with their own systems.

In summary, classical procurement methods in SMEs create operational drag and undermine competitiveness in a digital economy. Until affordable, localized automation tools become widely available, SMEs in Kazakhstan will remain at a disadvantage both domestically and internationally (Sidorov, 2023).

1.3 Role of AI, ML, and Blockchain in Procurement

Technologies such as artificial intelligence (AI), machine learning (ML), and blockchain are playing an increasingly transformative role in procurement, particularly in enterprise-level digital ecosystems. These tools allow organizations to transition from reactive, manual processes to proactive, data-driven strategies. AI enables intelligent supplier selection based on parameters like price, delivery time, contract history, and quality ratings. It can also automate repetitive tasks such as document classification, purchase order creation, and approval workflows—freeing procurement professionals to focus on strategic activities.

Machine learning complements these capabilities by analyzing large volumes of procurement data to identify patterns and generate forecasts. It helps in demand planning,

cost prediction, and identifying optimal reorder points. ML models can dynamically adjust sourcing strategies based on seasonality, supplier reliability, and market trends. Over time, these algorithms become more accurate, allowing companies to continuously optimize purchasing decisions with minimal human intervention.

Blockchain, on the other hand, addresses one of procurement's longstanding challenges: trust and transparency. With its decentralized and immutable ledger system, blockchain enables tamper-proof contract storage, verifiable delivery logs, and secure multi-party transactions. In supply chains where multiple entities are involved—such as suppliers, logistics providers, and customs authorities—blockchain ensures that all stakeholders have access to the same version of the truth. This significantly reduces fraud, disputes, and audit complexity.

These technologies are already being integrated into modern ERP systems used by large enterprises. For instance, SAP and Oracle have incorporated AI-driven supplier scoring, ML-based spend analytics, and blockchain-backed smart contracts into their platforms. As a result, enterprises can manage procurement holistically across departments and geographic regions.

However, among small and medium-sized enterprises (SMEs) in Kazakhstan, the implementation of these advanced technologies remains rare. According to Sidorov (2023), key barriers include:

- Lack of access to user-friendly procurement platforms that integrate AI/ML features;
- Limited technical expertise within SME teams to manage, interpret, or even deploy such systems;
- Low awareness of the strategic value of automation and data analytics;
- Absence of scalable infrastructure (e.g., cloud hosting, API ecosystems) in many regional areas;
- No localized tools adapted to the specific legal, financial, and linguistic realities of Kazakhstan.

Most existing solutions that offer AI or blockchain-based features are designed for large corporations and are either prohibitively expensive or too complex for SMEs to adopt. These platforms often require significant customization, long onboarding processes, and ongoing technical support—resources that most Kazakhstani SMEs cannot afford.

Moreover, even when cloud-based global solutions are technically accessible, they rarely support integration with Kazakhstani payment gateways, government databases, or tax-compliant documentation formats. This makes them unsuitable for companies operating in regulated sectors or dealing with local suppliers and clients.

In such an environment, advanced procurement technologies remain aspirational for most SMEs in Kazakhstan. The promise of AI, ML, and blockchain can only be realized if embedded within an accessible, localized system—one that simplifies rather than complicates the procurement process, while still providing long-term value. A platform tailored specifically for the Kazakhstani SME ecosystem, incorporating intuitive AI-powered tools and the potential for blockchain-based document validation, could help bridge this innovation gap and lay the foundation for smart procurement across the country, as Sidorov (2023) notes.

1.4 Challenges in Digital Procurement

Despite the growing recognition of procurement automation as a catalyst for operational efficiency, SMEs in Kazakhstan encounter a range of systemic, infrastructural, and behavioral barriers that hinder the adoption of digital tools. While the benefits of automation—such as faster transactions, improved data integrity, and transparency—are well-documented, the local context presents unique challenges that prevent many businesses from transitioning away from manual methods.

One of the most pressing issues is **low digital literacy**, particularly among SMEs operating in rural or semi-urban areas. Business owners and staff in these regions often lack formal IT training and have limited experience with cloud platforms, e-commerce systems, or digital financial tools. Without foundational skills, even simple automation tools can seem intimidating, leading to a reliance on traditional pen-and-paper workflows or basic communication apps like WhatsApp and Excel for procurement-related tasks.

Lack of trust in online platforms is another persistent barrier. Many SMEs express concerns about sharing financial data, contract information, and supplier records on digital platforms—especially those developed by foreign vendors. This hesitation is rooted in past experiences with fraud, poor user support, or fears of non-compliance with local regulatory requirements. Financial transactions, in particular, are viewed as high-risk if not processed through trusted, Kazakhstan-based systems.

Additionally, **there is no unified, centralized B2B procurement platform** in Kazakhstan that caters specifically to SME needs. Existing solutions tend to either

target large enterprises or operate as generalized e-commerce platforms that lack key procurement functionalities such as:

- tax-compliant invoicing in accordance with Kazakhstani law,
- bilingual interfaces (Kazakh/Russian),
- localized logistics options and pricing models,
- built-in support for digital signatures and fiscal documentation.

The high cost and complexity of international platforms further isolates SMEs from accessing global solutions. Platforms like SAP Ariba or Oracle Procurement Cloud are not only financially out of reach for small businesses but also require extensive customization and IT support to function properly within the Kazakhstani legal and infrastructural framework. As a result, even SMEs that wish to digitize their procurement are often forced to choose between overpaying for tools they cannot fully use or continuing to work manually.

Compounding this is the **fragmented nature of Kazakhstan's digital infrastructure**. Procurement does not exist in isolation—it requires seamless integration between suppliers, logistics providers, payment systems, tax authorities, and regulatory frameworks. Currently, these components operate in silos, and SMEs must manually coordinate between them. For example, ordering goods through one platform may require processing payments through an unrelated bank portal, submitting invoices via email, and generating contracts in Word. This lack of integration negates the potential benefits of automation by increasing administrative burden rather than reducing it.

A study by Ivanov & Petrov (2023) highlights similar issues within **public procurement systems**. Although Kazakhstan has introduced electronic tender platforms to improve access for SMEs, many businesses still struggle with registration procedures, complex interfaces, and certification requirements. These barriers are particularly high for newly established firms and businesses outside of major urban centers. The study emphasizes that these obstacles disproportionately affect SMEs due to their limited human and financial resources.

Importantly, the same structural barriers identified in public procurement are mirrored in **private-sector B2B procurement**, where no robust, localized systems currently exist. While some SMEs have adapted by building informal supplier networks or using consumer-grade tools, these workarounds are inherently unsustainable and introduce risks such as undocumented purchases, inconsistent pricing, and poor supply reliability.

In this environment, the lack of a localized, affordable, and intuitive B2B procurement system becomes a significant blocker to the digital transformation of the SME sector. Addressing this gap will require more than just introducing new software—it demands a platform that is **embedded in local business practices**, **compliant with national regulations**, and **designed for non-technical users**. Only then can SMEs begin to reap the full benefits of automation and position themselves as competitive players in Kazakhstan's emerging digital economy, as Ivanov and Petrov (2023) conclude.

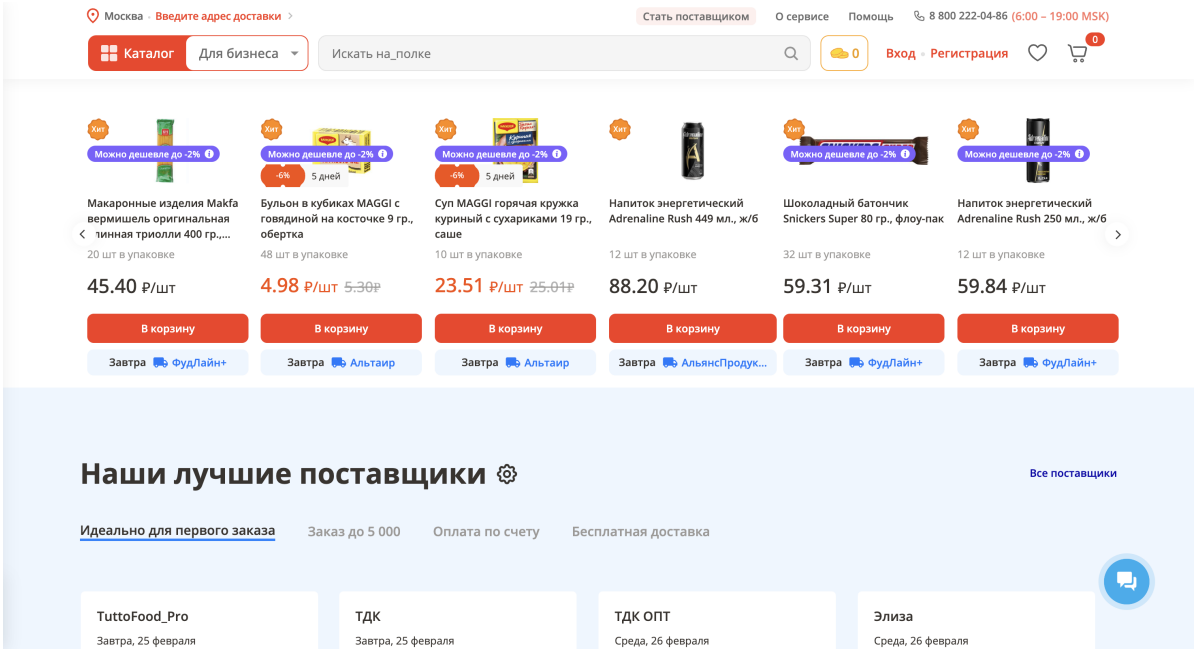
1.5 Analysis of Existing Systems

1.5.1 Case Study: *napolke.ru*

The Russian B2B platform *napolke.ru* offers a valuable example of what a semi-automated procurement environment can look like when designed for wholesale purchasing. The system includes key features such as:

- Automated supplier and product categorization
- Real-time price comparison across vendors
- Dynamic inventory tracking
- Regional logistics integration
- A unified cart that supports multi-vendor checkout
- Order segmentation per supplier and automatic delivery planning

These functions provide clear benefits for buyers, particularly small retailers and wholesalers, by reducing friction in procurement workflows. The interface is user-friendly and designed to handle large product volumes, making it efficient for business users.



Despite its functional maturity, however, *napolke.ru* is not suitable for direct implementation in Kazakhstan. Its architecture and legal assumptions are closely tied to the Russian economic environment, which poses several compatibility issues for Kazakhstani users:

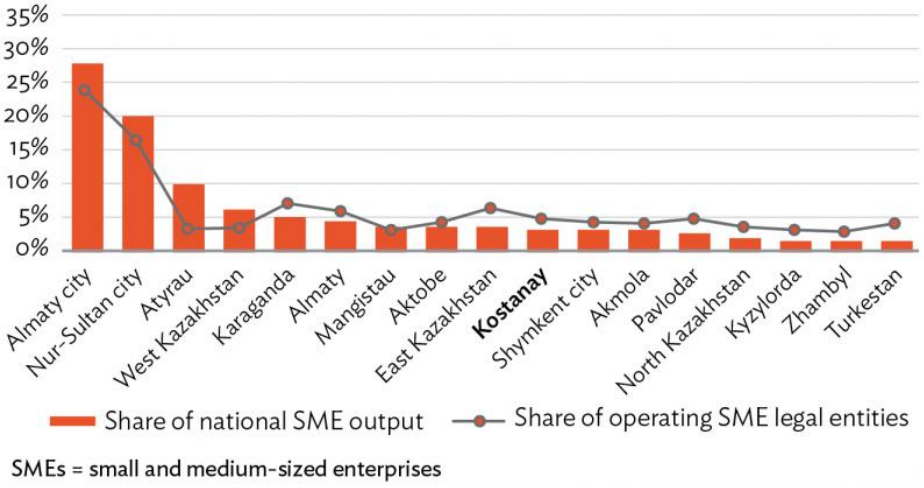
- **No support for KZT (Kazakhstani tenge)**: All transactions are conducted in Russian rubles, requiring currency conversion and complicating financial records.
- **No integration with Kazakh payment infrastructure**: Platforms such as Kaspi, FortePay, and Halyk are not supported, nor are any local gateways like Fondy.
- **Incompatibility with Kazakh tax and invoicing standards**: There is no built-in functionality to generate invoices compliant with Kazakhstan's VAT and electronic documentation rules.
- **No Kazakh language support**: The platform is exclusively in Russian, limiting accessibility and excluding users who prefer or require Kazakh.

Therefore, even though *napolke.ru* successfully demonstrates how B2B automation can work in practice, its geographic and regulatory scope makes it unsuitable for replication in the Kazakhstani context without significant and costly modification. It also lacks features that would specifically benefit local SMEs, such as support for government procurement compliance, bilingual UX, or regional supplier databases.

Its limitations underline the broader lack of platforms tailored to Kazakhstan's business and legal environment, especially in the procurement space. The potential value that *napolke.ru* offers further reinforces the need for a Kazakhstani equivalent designed from the ground up for local SMEs and producers.

1.5.2 Gap Analysis for Kazakhstan Market

While advanced procurement platforms exist abroad, Kazakhstan continues to face a persistent infrastructure and digital services gap, especially when it comes to empowering small businesses outside major economic centers. The Asian Development Bank (2020) provides a clear picture of this disparity.



According to the data, cities like Almaty and Astana (Nur-Sultan) dominate the national economic contribution of SMEs, with robust infrastructure, access to skilled labor, and dense supply chains. In contrast, regions such as Kostanay, Turkestan, and Zhambyl have a large number of registered SMEs but contribute minimal actual output to the national GDP. This disconnect between registration and productivity suggests that many businesses exist only on paper or operate well below their potential capacity.

Several structural factors contribute to this underperformance:

- **Lack of digital procurement channels**: SMEs in peripheral regions have little to no access to centralized supplier networks or procurement marketplaces.
- **Limited competition among suppliers**: In smaller cities, businesses often rely on a narrow circle of known vendors, leading to inflated prices and limited choice.

- **Absence of contract and document automation tools**: Procurement contracts, delivery records, and acceptance certificates are managed manually, creating legal and logistical friction.
- **Logistical isolation**: Many regional SMEs struggle with delivery coordination, especially when sourcing from suppliers based in urban hubs.

Without digital tools to streamline sourcing, pricing, documentation, and supplier discovery, SMEs in these regions remain economically inactive or inefficient. The result is not only reduced competitiveness but also limited ability to scale or integrate into national and regional supply chains.

Implementing a localized procurement platform specifically designed for Kazakhstan would address these bottlenecks. Such a platform would:

- Digitally connect SMEs to a verified supplier base across Kazakhstan;
- Automate contract generation, ensuring compliance with national tax and legal frameworks;
- Simplify logistics coordination using embedded delivery APIs or third-party integrations;
- Enable use of local payment systems and tenge billing;
- Offer full bilingual support (Kazakh/Russian);
- Provide procurement analytics and reports for smarter business planning.

More importantly, it would activate dormant businesses by lowering the barriers to professional procurement—especially in rural areas where formalization and automation are almost nonexistent. SMEs that currently rely on phone-based orders and handwritten notes could transition to digital ordering, standardized pricing, and transparent documentation—all key ingredients for sustainable growth.

As the government continues to push digital transformation across sectors, addressing this procurement gap becomes both an economic and strategic priority. A domestic solution would support not just buyers, but also emerging Kazakhstani manufacturers and micro-suppliers, who currently have no structured channel to reach national business customers.

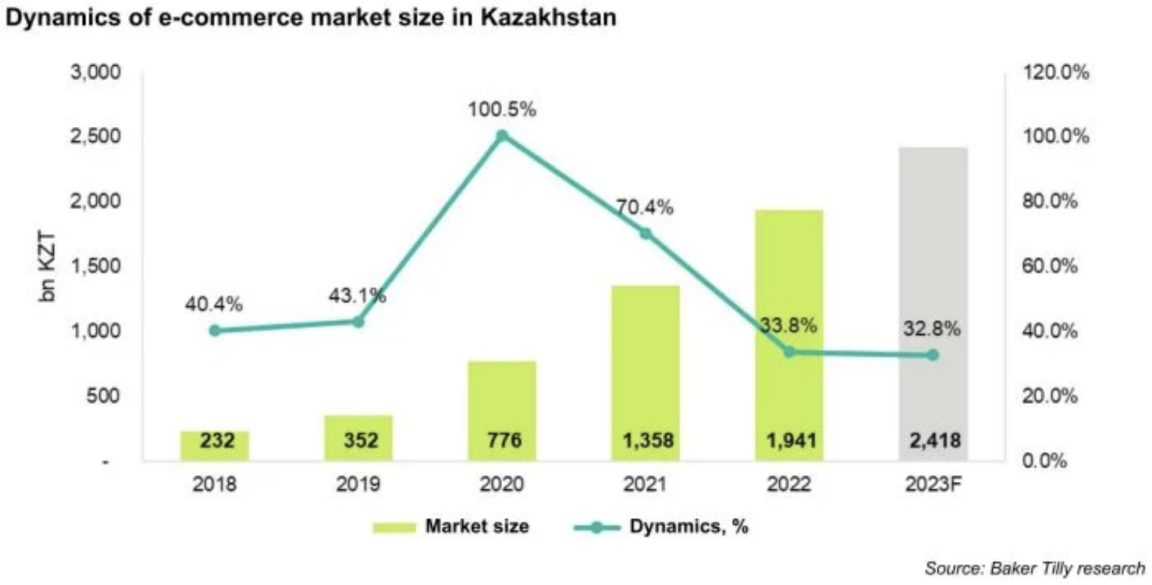
1.5.3 Functional Comparison Table

Functionality	**napolke.ru / Global Platforms**	**Kazakhstani SME Needs**
Support for national currency (KZT)	No	Yes
Local tax-compliant invoicing	No	Yes
Integration with local banks	No	Yes
Electronic Signature (EDS) support	No	Yes
Kazakh language interface	No	Yes
Logistics tailored to Kazakhstan	No	Yes
SME-friendly UI/UX	Partial	Yes

Таблица 1.1: Functional gap between global platforms and local SME needs

1.5.4 Growth of E-Commerce as a Foundation for B2B Platforms

Over the past five years, Kazakhstan's e-commerce sector has undergone a period of rapid and sustained expansion. According to Baker Tilly (2023), the volume of the national e-commerce market grew from **KZT 232 billion in 2018** to **KZT 1.94 trillion in 2022**, with projections reaching **KZT 2.4 trillion by the end of 2023**.

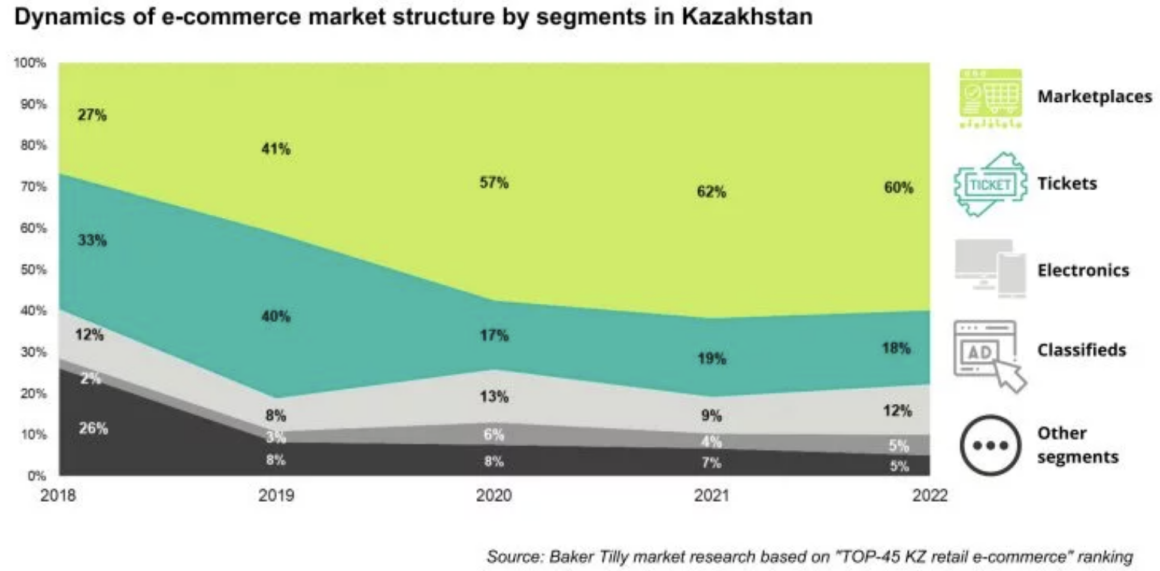


This exponential growth has been driven by multiple factors:

- Increased internet penetration and mobile usage

- Widespread adoption of digital payments
- Improved logistics infrastructure across key regions
- Government incentives and digitalization initiatives
- Accelerated online buying behavior following the COVID-19 pandemic

Perhaps more importantly, the **structure** of the e-commerce market has fundamentally changed. In 2018, a significant portion of online trade was handled through *classifieds platforms and independent e-shops*. By 2022, however, *marketplaces had come to dominate*, representing **60% of all online transactions**, while the share of classifieds and standalone stores declined sharply.



Source: Baker Tilly market research based on "TOP-45 KZ retail e-commerce" ranking

This shift indicates that users—both consumers and sellers—prefer centralized, user-friendly platforms that offer secure transactions, integrated logistics, and a broad product base. These marketplaces reduce friction, build trust, and streamline operations for all parties involved. This trend is a clear signal of **digital maturity and readiness for platform-based business models** in Kazakhstan.

Yet despite this impressive progress in the B2C (business-to-consumer) space, the **B2B (business-to-business)** segment—particularly in the field of procurement and contract lifecycle management—**remains vastly underserved**. There are currently no comprehensive digital platforms in Kazakhstan that enable small and medium-sized enterprises (SMEs) to conduct end-to-end procurement operations online.

Unlike consumers, businesses have complex purchasing needs that go beyond selecting products and placing orders. SME procurement involves:

- Multi-supplier price comparisons
- Tax-compliant invoice and contract generation
- Coordination with logistics providers
- Order segmentation and tracking
- Delivery confirmation and formal acceptance
- Payment through localized financial gateways
- Compliance with government accounting and reporting standards

While B2C marketplaces have succeeded by focusing on individual buyers, none of the existing platforms offer this kind of structured, workflow-oriented procurement experience tailored to businesses. This creates a clear **disconnect** between Kazakhstan's overall digital potential and the current state of SME procurement infrastructure.

Moreover, this gap is not simply technological—it represents a missed economic opportunity. SMEs make up a substantial portion of the country's business landscape and play a vital role in local production, distribution, and employment. However, many of these businesses remain disconnected from the benefits of digitization due to the absence of a procurement platform designed for their operational realities.

The continued dominance of **manual procurement methods**—as described in Sections 1.2 and 1.4—only compounds the problem. Businesses that already use digital tools for sales and payments are ready and willing to adopt procurement solutions, provided they are localized, intuitive, and affordable.

A dedicated B2B procurement platform could:

- Digitally link SMEs with verified local suppliers
- Reduce transaction costs and pricing inconsistencies
- Enable regional producers to access national markets
- Automate documentation and improve legal compliance
- Create new opportunities for digital logistics, tax accounting, and supplier financing

Given the infrastructure, behavior, and trust already established by the B2C e-commerce boom, the foundation has clearly been laid for B2B transformation. The lack of such a system in Kazakhstan is not due to a lack of demand—it is due to a lack of targeted, purpose-built tools.

As such, developing a **universal, Kazakhstani-focused B2B procurement platform** is not only technically feasible—it is strategically necessary. It would empower a new generation of digitally integrated SMEs, drive regional economic activation, and strengthen national supply chains across every sector.

2 Methodology

This chapter outlines the research methodology applied to explore the current state of procurement processes among small and medium-sized enterprises (SMEs) in Kazakhstan, with a focus on evaluating the feasibility and necessity of implementing an automated procurement platform. The research design was constructed to provide both depth and breadth of insight through a combination of **theoretical foundations** and **empirical data collection**. The study involved two major components:

1. A **structured literature review** (discussed in Chapter 1), which establishes the theoretical and contextual backdrop for procurement automation, digital transformation in SMEs, and comparative technology frameworks.
2. An **original quantitative survey**, designed to gather practical data directly from SME decision-makers regarding their operational procurement methods, challenges, and attitudes toward digital solutions.

2.1 Research Design and Approach

The study adopted a **mixed-methods approach**, which allowed for integration of both qualitative and quantitative data. The literature review provided conceptual grounding by synthesizing academic and industry sources concerning procurement automation, SME development, and digital infrastructure trends in Kazakhstan and abroad.

The **empirical component** involved a structured survey conducted among SME decision-makers—owners, managers, and procurement staff—who were actively involved in purchasing decisions.

The objective of this approach was to:

- Understand current procurement practices and workflows;
- Identify key pain points and inefficiencies in procurement;
- Assess the perceived importance of automation;
- Evaluate openness to adopting new procurement technologies.

This dual-track methodology helped ensure both **contextual relevance** and **practical applicability** of the study's outcomes.

2.2 Data Collection and Survey Design

A **quantitative online survey** was selected as the primary instrument for collecting empirical data. The questionnaire was distributed via Google Forms and shared through professional networks, SME-focused forums, and email groups. Responses were collected over a two-week period, resulting in a total of **50 complete and valid submissions**.

2.2.1 Questionnaire Structure

The questionnaire included multiple-choice and Likert-scale questions across the following five sections:

1. Business Type and Company Size

Respondents identified the type of business they run. As shown in Figure 2.1, a majority of businesses (68%) operate in retail trade, followed by wholesale (20%), manufacturing (6%), and services (6%).

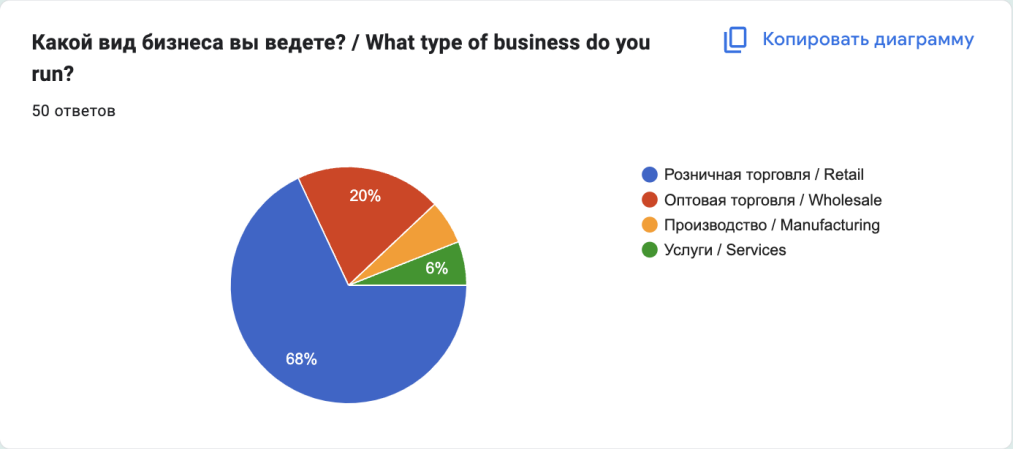


Рис. 2.1: What type of business do you run?

Company size distribution is shown in Figure 2.2, where most businesses (66%) reported employing between 6–20 people. Only 12% of respondents reported having over 20 employees.

Рис. 2.2: How many employees does your company have?

2. Procurement Practices

Participants were asked how procurement is managed. As seen in Figure 2.3, 80% of companies conduct procurement independently, with staff manually sourcing suppliers. Only 8% use specialized platforms.

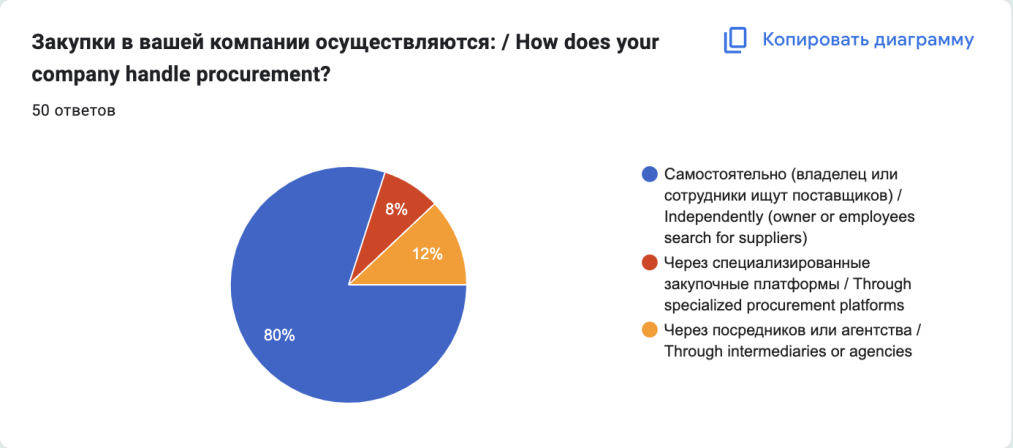


Рис. 2.3: How does your company handle procurement?

3. Procurement Challenges

Respondents highlighted the difficulties they face when procuring goods. The most common issues were long supplier search processes (44%), contract management complexity (38%), pricing opacity (34%), delivery delays (32%), and errors in calculations (44%). These are illustrated in Figure 2.4.

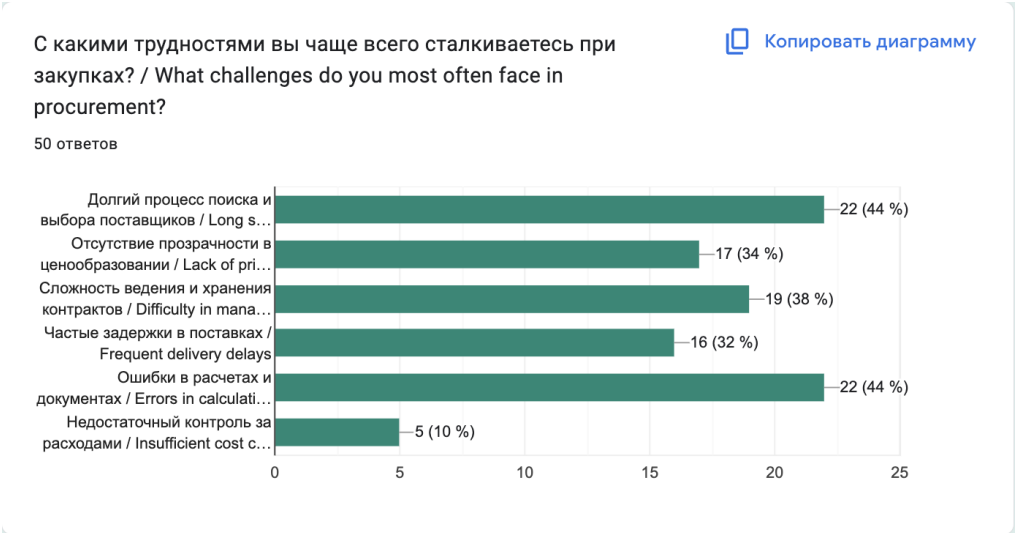


Рис. 2.4: What challenges do you most often face in procurement?

4. Importance of Procurement Automation

In Figure 2.5, it is clear that automation is a priority for SMEs—48% rated its importance at 4 out of 5, and 36% gave it the highest rating of 5.

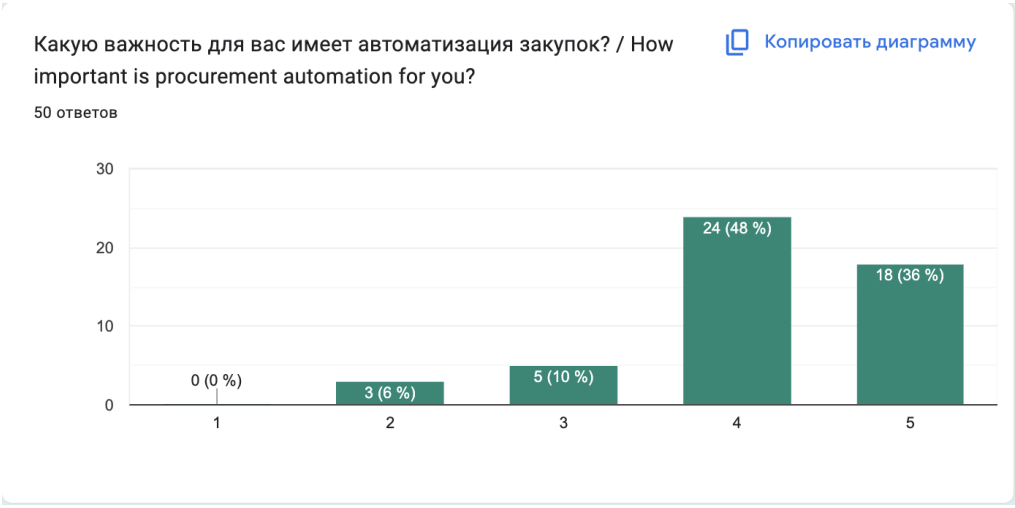


Рис. 2.5: How important is procurement automation for you?

5. Willingness to Adopt a New Platform

Finally, Figure 2.6 shows that 66% of participants expressed strong interest in testing a new platform, and another 26% would consider trying it if it solved their procurement problems.

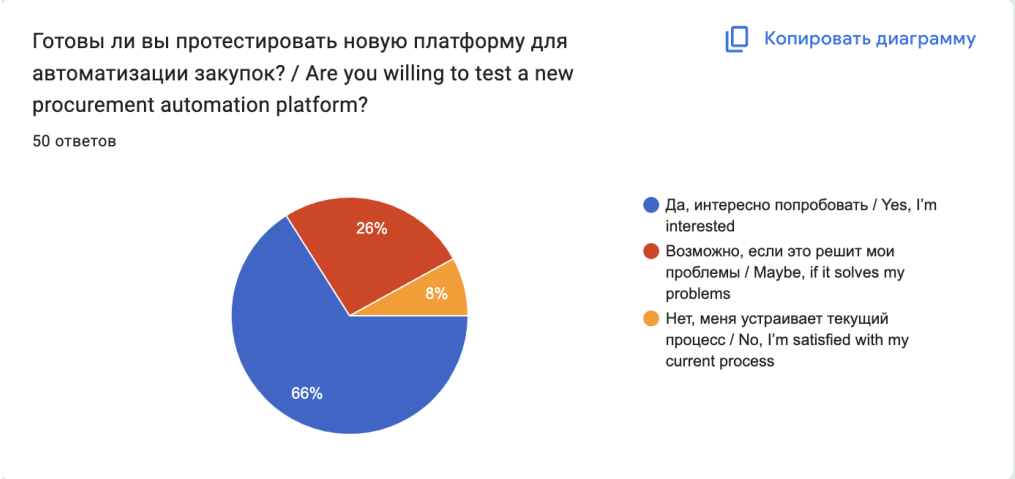


Рис. 2.6: Are you willing to test a new procurement automation platform?

The **bilingual survey (Russian and English)** helped eliminate language barriers and ensured broader participation across Kazakhstan's diverse SME community. The question structure combined closed-ended responses with scaled options to allow both quantitative aggregation and insight into behavioral trends.

2.2.2 Key Findings

The survey results provide strong empirical evidence of the challenges and unmet needs SMEs face in the area of procurement. Each thematic block in the questionnaire revealed patterns that are critical for both academic understanding and system design.

- **Decentralized procurement responsibilities**: As indicated in Figure 2.3, 80% of respondents rely on internal manual processes, typically involving either the business owner or an employee. This confirms that procurement duties in SMEs are not usually assigned to dedicated specialists, which leads to fragmented workflows and inconsistent decision-making.

- **Supplier discovery and evaluation are inefficient**: Nearly half of the participants reported that finding and selecting suppliers is time-consuming (44%). Without digital directories, filtering mechanisms, or reputation tracking, many SMEs waste time evaluating offers individually.

- **Human error remains a core issue**: Also cited by 44% of respondents were frequent errors in procurement-related calculations and documentation, including

mismatches in pricing, order volume, or tax values. This implies an urgent need for automation in document generation and invoice verification.

- **Automation is widely seen as valuable**: With 84% of businesses rating procurement automation as "important" or "very important" (Figure 2.5), the data clearly confirms that digital tools are not only welcomed, but actively desired by SMEs—particularly in environments where small teams must handle complex supplier interactions.

- **Adoption potential is high**: As Figure 2.6 shows, 66% of respondents are ready to test a new system, with another 26% open to trying it if their core problems are addressed. This means that more than 90% of respondents are potentially reachable during platform onboarding, assuming functionality and usability meet expectations.

These findings validate the hypothesis that there is a critical gap between SME needs and the digital tools currently available on the market, especially in Kazakhstan's local context.

2.3 Validity and Reliability of Data

Ensuring the **validity** and **reliability** of the data was a priority throughout the study. Several techniques were applied to enhance the integrity of the results:

- **Sampling relevance**: The survey was distributed among verified SME channels, including professional groups, chambers of commerce, and industry-specific online communities. This ensured that respondents were relevant stakeholders who actively participate in procurement decisions.

- **Bilingual accessibility**: All questions were presented in both Russian and English to accommodate Kazakhstan's linguistic diversity and reduce potential misunderstanding. This increased inclusiveness and minimized language bias.

- **Neutral phrasing**: Care was taken to phrase each question in a non-leading way, allowing respondents to express their real views without being influenced by suggestive wording.

- **Standardized distribution**: The survey format and order of questions were identical for all participants, ensuring that all respondents encountered the same conditions and logic.
- **Consistency checks**: All submissions were manually screened to detect duplicate responses or logical inconsistencies (e.g., businesses claiming to use automation but also listing manual-only practices). No conflicting cases were detected, indicating a high level of **internal reliability**.

Although the sample size ($n = 50$) is relatively small, it is appropriate for an **exploratory study** aimed at understanding feasibility and system requirements at the MVP (Minimum Viable Product) stage. The uniformity of answers across respondents also strengthens the reliability of the identified trends.

2.4 Implications for Platform Design

The insights gained through this study provide direct and actionable guidance for the design of the procurement automation platform. Based on user behavior, expectations, and limitations, the system must address the following five core dimensions:

1. **Simplicity and Usability**
   Given that most SMEs do not have full-time procurement specialists, the interface must be intuitive and require minimal training. Navigation should prioritize ease-of-use and avoid unnecessary complexity.
2. **Automated Supplier Selection**
   To solve the top-cited problem—time-consuming supplier searches—the platform should feature a built-in supplier database with smart filters and ranking algorithms that help users quickly identify reliable vendors.
3. **Contract and Document Management**
   One of the major pain points identified was difficulty managing contracts. The platform must offer auto-generated purchase agreements, acceptance acts, and invoice templates with proper tax formatting.
4. **Transparency and Price Comparison**
   To address the lack of pricing clarity (34%), real-time product listings, side-by-side price views, and order segmentation by supplier must be implemented.

5. **Scalability and Integration**
   The system must be compatible with Kazakhstani payment systems (e.g., Kaspi, Halyk, Forte), support billing in KZT, and comply with local legal requirements—including electronic digital signature (EDS) support.

These features will not only improve procurement efficiency for SMEs, but also strengthen regional supplier visibility and unlock new distribution opportunities for emerging Kazakhstani manufacturers and wholesalers.

3 System Design and Architecture

This chapter provides a comprehensive overview of the backend architecture, design decisions, data structures, inter-module interactions, and implementation specifics of the system. The backend platform serves as the core engine of a B2B procurement automation system, designed using Domain-Driven Design (DDD) principles and implemented in Go. It combines monolithic deployment simplicity with modular organization, providing scalability, security, maintainability, and readiness for microservice transition.

3.1 System Overview and Module Structure

The backend of the system is structured as a **modular monolith**, meaning the application is compiled and deployed as a single executable unit while internally structured into independent modules based on business domains. This architecture ensures high performance, simplified deployment, and maintainable code organization while laying a foundation for future microservices migration if necessary.

Each module in the system corresponds to a **bounded context** defined by the principles of Domain-Driven Design (DDD). These contexts encapsulate all the functionality and business logic related to a particular domain, helping developers maintain separation of concerns and align the codebase closely with business terminology.

Key Characteristics of the Modular Monolith

- **Logical Isolation**: All modules are physically part of the same binary but logically separated by package boundaries.
- **Tight Internal Cohesion**: Within each module, components are highly cohesive and communicate through well-defined interfaces.
- **Loose Inter-Module Coupling**: Modules interact through Client interfaces to reduce dependencies and improve testability.
- **Centralized Configuration and Database**: Shared services such as database connection, logging, and configuration are maintained centrally.

Core Modules and Their Responsibilities

Auth Module
- Manages user authentication (login/register) and token issuance.
- Uses bcrypt for password hashing and JWTs for secure session handling.
- Secures routes with middleware validation.

User Module
- Stores user profiles, including name, phone number, and contact details.
- Exposes endpoints for viewing and updating profile data.

Product Module
- Manages product catalog data, including names, images, GTINs, and supplier-specific pricing.
- Provides search and filtering capabilities.
- Maintains product-to-supplier relationships.

Cart Module
- Enables customers to add, remove, and modify product selections.
- Groups items by supplier and dynamically calculates delivery thresholds, total cost, and tax information.
- Supports cart cleanup and checkout triggering.

Order Module
- Handles conversion of cart data into confirmed orders.
- Maintains state transitions: Pending, In Progress, Completed, Cancelled.
- Allows customers to cancel unconfirmed orders and suppliers to update order statuses.

Contract Module
- Automatically generates delivery contracts for each supplier involved in an order.
- Manages digital signatures and maintains contract lifecycle states.
- Interfaces with document generation and future e-signature modules.

Supplier Module
- Enables suppliers to configure business logic including delivery fees, minimum order amounts, and tax registration.
- Provides views to manage incoming orders and monitor performance metrics.

3.1.1 Module Internals

Each module is further divided into internal layers:

- **Handler**: REST API controllers using HTTP routers to map endpoints to service methods.
- **Service**: Business logic and use-case orchestration, validation, and transformation.
- **Repository**: Database interface layer using SQL queries or ORM models to interact with PostgreSQL.
- **Model**: Domain objects and DTOs defining structure and validation logic.
- **Client**: Interface contracts to interact with other modules (e.g., OrderClient, ProductClient).

This design ensures that changes in one module do not cascade unpredictably across the system, improving stability and reducing the cost of future updates.

The backend is organized as a modular monolith, a practical and scalable architectural choice for growing platforms. Each domain-specific functionality is encapsulated in a logically isolated bounded context. This organization ensures modularity while keeping inter-module communication simple and performant through in-process method calls.

Summary of Key Modules and Responsibilities

- **Auth Module**: Handles user registration, login, JWT token issuance, and password hashing.
- **User Module**: Manages personal profile data, including name and phone number updates.
- **Product Module**: Contains logic for catalog management, GTIN handling, pricing, and supplier relationships.
- **Cart Module**: Aggregates selected products by supplier, performs price calculations, and applies business rules such as delivery thresholds and minimum order amounts.
- **Order Module**: Handles creation, cancellation, and state transitions of orders (e.g., Pending $\rightarrow$ In Progress $\rightarrow$ Completed).

- **Contract Module**: Automates creation of delivery agreements, manages e-signature flows, and tracks fulfillment.
- **Supplier Module**: Allows suppliers to configure business logic, including minimum order amounts and delivery pricing.

Each module includes a handler layer (REST endpoints), a service layer (business logic), and a repository layer (data access abstraction).

3.2 System Architecture Description

The architecture of the backend system is designed with a strong adherence to clean architecture principles and Domain-Driven Design (DDD), ensuring a clear separation of concerns and strict modular boundaries. The system follows a layered structure to allow independent development, testing, and scaling of its components.

At its core, the backend operates as a extbfmodular monolith—a single deployable unit where each bounded context is isolated by package, logically self-contained, and communicates via well-defined interfaces. This ensures rapid development and simplified deployment while remaining ready for future migration to microservices.

3.2.1 Layered Architecture Breakdown

- **Handler (API Layer)**
  - Receives HTTP requests and routes them to appropriate services
  - Performs JSON decoding, input validation, and response formatting
  - Applies middleware (authentication, logging)
  - Example routes: exttt/api/cart/checkout, exttt/api/contract/sign
- **Service Layer**
  - Orchestrates business logic and domain use cases
  - Handles workflows such as supplier validation, price calculations, and cart aggregation
  - Initiates cross-module interactions via Client interfaces
  - Ensures domain rules and invariants are respected

- **Repository Layer**
  - Interacts with the PostgreSQL database using SQL or ORM
  - Encapsulates all persistence logic for better testability
  - Supports dependency injection for mock database testing
- **Model Layer**
  - Houses core business objects and data transfer objects (DTOs)
  - Defines validation rules, constraints, and schemas
  - Separates transport-layer and domain-layer structures
- **Client Layer**
  - Interfaces for communication between modules (e.g., extttProductClient, extttOrderClient)
  - Abstracts dependencies to maintain encapsulation
  - Facilitates modular testing and future migration to services

3.2.2 Inter-Module Communication

Modules interact exclusively through defined Client interfaces. For example, the Cart module uses the ProductClient to retrieve product details without directly accessing internal logic of the Product module. This reduces tight coupling and supports scalability.

3.2.3 Architecture Benefits

- **Loose Coupling**: Interfaces and module boundaries minimize dependencies
- **High Testability**: Layers and interfaces support effective unit testing
- **Scalability**: Domain-aligned boundaries allow horizontal scaling
- **Security**: Middleware handles authentication and HTTPS ensures secure communication

3.2.4 Security Mechanisms

- **JWT Authentication**: Stateless, signed token-based session control
- **Middleware Enforcement**: Ensures protected route access
- **Role-Based Access Control**: Differentiates customers and suppliers

3.2.5 Transport Protocol

All API requests and responses use JSON over HTTPS. External clients interact via REST APIs, with full documentation available via Swagger.

3.2.6 Responsibilities

- **Handler Layer**: Parses HTTP requests, validates input, and formats JSON responses.
- **Service Layer**: Implements core business logic and enforces rules.
- **Repository Layer**: Manages SQL and database operations.
- **Model Layer**: Defines domain entities and DTOs.
- **Client Layer**: Facilitates inter-module interaction through interfaces.

The backend's layered and modular architecture ensures that domain logic is preserved and isolated from technical infrastructure. This design supports maintainability, extensibility, and operational stability.

3.3 Architecture Diagrams

The architecture of the backend system is not only reflected in its layered design, but also reinforced through visual diagrams that aid in both development and stakeholder communication. These diagrams illustrate the flow of procurement processes, module interaction, and structural design of the data layer.

3.3.1 BPMN Procurement Flow

The BPMN (Business Process Model and Notation) diagram demonstrates the lifecycle of a procurement process. It maps out the decision-making flow from the submission of a procurement request to supplier confirmation, contract drafting, order fulfillment, and performance evaluation. This diagram clearly distinguishes between valid and invalid requests, approval steps, contract signing, and automated actions like reminders and rating updates.

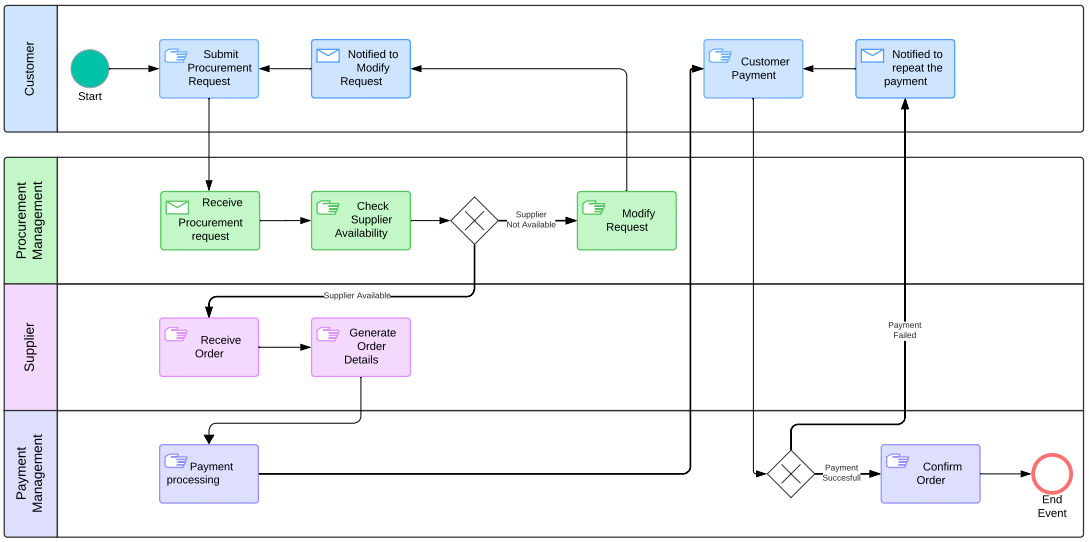


Рис. 3.1: BPMN Procurement Process Flow

3.3.2 UML Activity Diagram

The UML activity diagram captures how procurement requests progress through conditional flows. It details interactions between stakeholders (buyer, supplier), outlines contract validation steps, and visualizes potential loops (e.g., request modification) and terminations. This diagram is critical for understanding how asynchronous steps (such as contract approval or rejection) are managed.

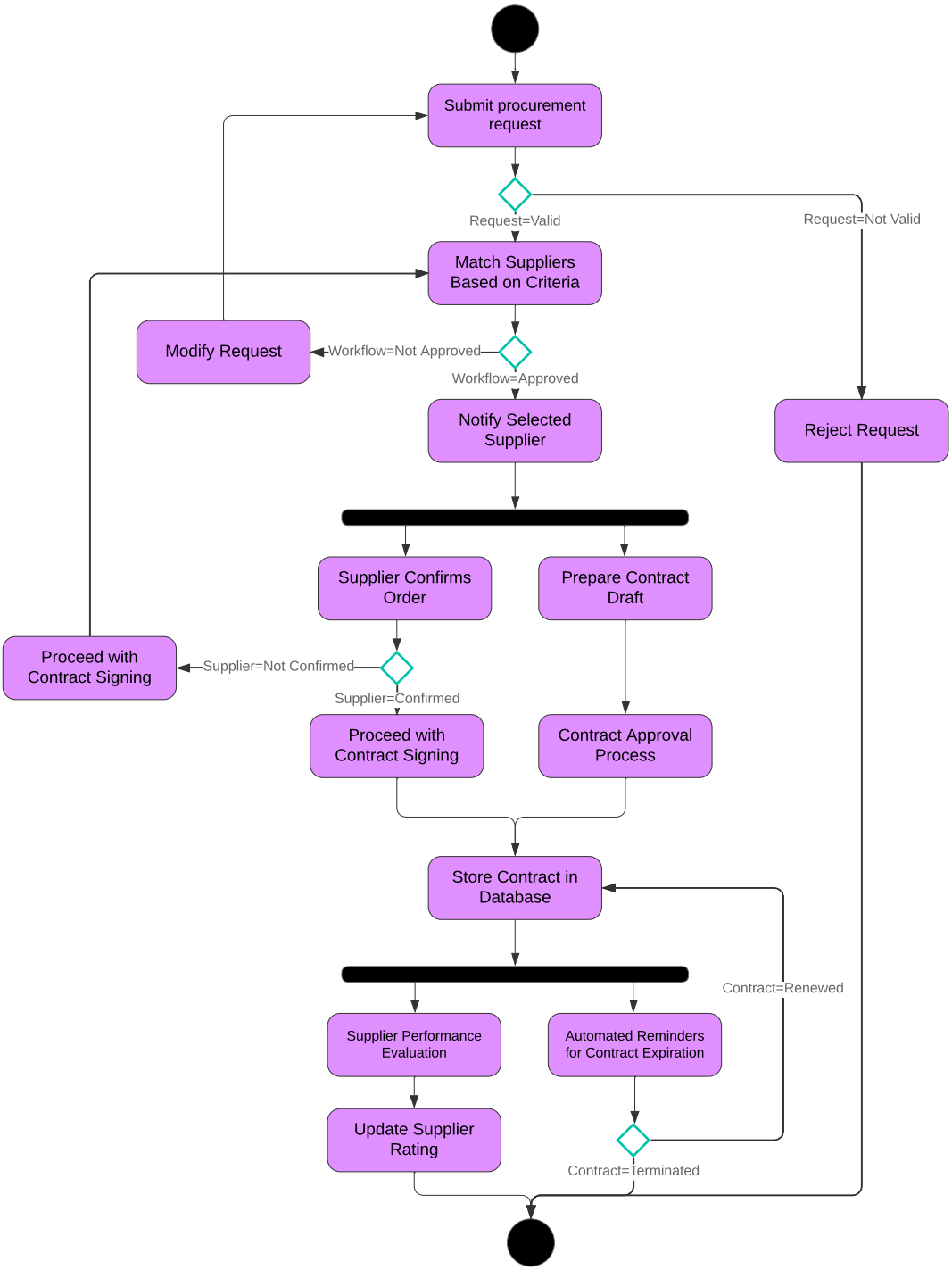


Рис. 3.2: UML Activity Diagram for Procurement Request Flow

3.3.3 Folder Structure Snapshot

To better understand how the system is physically organized in the file system, a simplified structure view is provided. It reveals the separation between internal infrastructure code and business-domain modules.

```
/cmd                    // Entry points
/internal
    /app                // Bootstrap and lifecycle logic
    /config             // Environment config loading
/modules
    /auth               // User login, JWT, password hashing
        /handler
        /service
        /repository
        /model
        /middleware
        /jwt
    /cart               // Redis-backed cart logic
    /order              // Order tracking and state updates
    /contract           // Contract management and signature
    /supplier           // Supplier dashboard, fees, logic
    /product            // Catalog and product-to-supplier logic
/pkg
    /client             // Inter-module contracts
    /db                 // PostgreSQL client, transactions
    /validator          // Input validation
```

3.4 Database Design (ERD and Table Descriptions)

The system employs PostgreSQL as the relational database management system, chosen for its reliability, ACID compliance, strong support for SQL standards, and robust indexing capabilities. The database schema is structured around normalized entities reflecting real-world business domains, ensuring both data integrity and future extensibility.

All domain models are carefully mapped to relational tables through explicit schema definitions. Foreign keys, indexes, constraints, and type definitions are applied rigorously to support transactional operations and maintain referential integrity.

Each core module (e.g., order, cart, product, supplier) owns its set of database tables, with relationships coordinated via foreign keys or join tables. This schema mirrors the Domain-Driven Design (DDD) structure used in the application codebase.

3.4.1 Entity-Relationship Diagram (ERD)

The ERD diagram outlines the structural relationships between the system's primary entities:

- `users`: Contains user profile and authentication metadata.
- `products`: Stores product catalog information, including name, GTIN, image, and base price.
- `suppliers`: Represents businesses offering products and services.
- `orders`: Captures user-submitted purchase requests with state-tracking fields.
- `contracts`: Links orders and suppliers, including timestamps and signing states.
- `cart_items`: Temporary storage for active carts prior to checkout.
- `delivery_conditions`: Defines supplier-specific delivery fees and minimum thresholds.

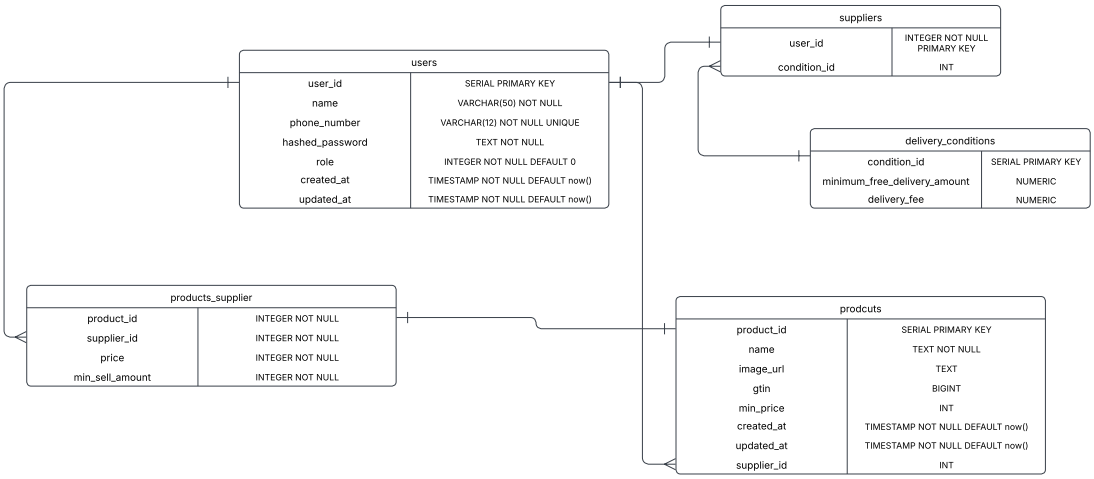


Рис. 3.3: Entity-Relationship Diagram (ERD)

The ERD highlights:

- One-to-many relationship from suppliers to products
- One-to-many relationship from users to orders
- Many-to-many simulated via join tables (e.g., orders with multiple suppliers)
- Foreign keys linking users, products, suppliers, and cart entries

3.4.2 Product Table Breakdown

The `products` table is one of the most critical in the system. It is used for rendering catalogs, price estimation, cart validation, and order composition.

- `product_id` (UUID): Unique identifier (Primary Key)
- `name` (TEXT, NOT NULL): Full display name
- `image_url` (TEXT): Optional reference to CDN-hosted image
- `gtin` (BIGINT): Global Trade Item Number for standardized goods
- `min_price` (DECIMAL): Minimum available price across suppliers
- `supplier_id` (UUID): Optional link to owner supplier
- `created_at` (TIMESTAMP): Entity creation time
- `updated_at` (TIMESTAMP): Last updated timestamp

prodcuts	
product_id	SERIAL PRIMARY KEY
name	TEXT NOT NULL
image_url	TEXT
gtin	BIGINT
min_price	INT
created_at	TIMESTAMP NOT NULL DEFAULT now()
updated_at	TIMESTAMP NOT NULL DEFAULT now()
supplier_id	INT

Рис. 3.4: Structure of the Products Table

Indexes are applied on `gtin`, `supplier_id`, and `name` fields to optimize query performance for catalog browsing and filtering.

Each product may be associated with multiple delivery options depending on its suppliers. These relationships are managed via a separate `products_supplier` table that enables price overrides, conditional availability, and tax classification.

This structured schema lays the foundation for transactional consistency, optimized queries, and clean reporting via SQL analytics, all while aligning with the domain logic of the Go application.

3.5 Security and Authentication

Security is a top priority in a platform handling sensitive procurement data, customer identities, and financial transactions. A layered security approach is applied to protect the system from unauthorized access, data leaks, and other threats at the infrastructure, application, and transport levels.

3.5.1 Authentication and Session Management

JWT-Based Authentication: All REST API endpoints are protected using JSON Web Tokens (JWT). Tokens are generated during login, include expiration timestamps, and are cryptographically signed with HMAC or RSA keys to prevent tampering.

Refresh Tokens (Planned): For enhanced session security, a refresh token mechanism is planned, allowing access tokens to be renewed securely without re-authentication.

Secure Password Hashing: User passwords are hashed using the bcrypt algorithm before being stored in the database, protecting user credentials from leakage.

3.5.2 Role-Based Access Control (RBAC)

Separation of Roles: The system enforces role-based policies to distinguish between end users (buyers) and suppliers. Suppliers can only access their product and order data, while customers can manage carts, profiles, and initiate orders.

Scoped Permissions: Route access is controlled via middleware that verifies user roles and scopes based on JWT claims.

3.5.3 Transport and Communication Security

HTTPS Everywhere: All traffic to and from the backend is encrypted via TLS (HTTPS), eliminating the risk of data interception over the network.

Same-Origin Policy and CORS Controls: The frontend and backend are configured with strict cross-origin resource sharing (CORS) policies to prevent unauthorized cross-domain API calls.

3.5.4 Middleware Enforcement

A custom authentication middleware is applied across modules to inspect JWTs, verify user identity, and attach user claims to request contexts. This middleware is integrated into the router stack and applied to every protected endpoint using layered route grouping.

3.5.5 Logging and Audit Trails

Login Attempts Logging: Successful and failed login attempts are logged with IP and timestamp metadata.

Order/Contract Events: Status changes, cancellations, and confirmations of orders and contracts are timestamped and stored for traceability.

Planned Logging Enhancements: In future iterations, tamper-proof audit trails and anomaly detection mechanisms will be introduced to flag suspicious activity.

Together, these mechanisms establish a secure foundation for building trust with users, meeting compliance requirements, and protecting sensitive business data throughout the lifecycle of a procurement transaction.

3.6 DevOps and Deployment

This section outlines how the backend system is developed, tested, built, and deployed in a consistent and scalable manner. To support high availability, maintainability, and fast iterations, the project uses modern DevOps tools and workflows.

3.6.1 Containerization

The entire application, including the Go backend and PostgreSQL database, is containerized using Docker.

- A `Dockerfile` is defined for building the backend image, specifying dependencies, environment variables, and startup commands.
- A `docker-compose.yml` configuration is used to orchestrate multi-container deployments locally and in staging environments.
- Docker networks are configured for internal module communication.
- Volumes are mounted for persistent PostgreSQL storage.

3.6.2 Continuous Integration and Delivery (CI/CD)

The project uses GitHub Actions (or GitLab CI) for continuous integration and delivery pipelines.

Key CI/CD stages include:

- **Build:** Compiles Go code and checks for compile-time errors.
- **Lint & Format:** Runs `golangci-lint` and formatting tools to ensure code style consistency.
- **Test:** Executes unit and integration tests.
- **Migrations:** Applies database migration scripts using `golang-migrate`.
- **Swagger Docs:** Auto-generates OpenAPI/Swagger documentation and publishes it at `/swagger/index.html`.
- **Deploy:** Pushes to a test or staging environment via Docker Compose or Kubernetes.

3.6.3 Monitoring and Logging

- **Structured Logging:** All logs use structured JSON format for easier aggregation and querying.
- **Centralized Log Aggregation:** Logs may be forwarded to Logstash, Fluentd, or Grafana Loki.
- **Prometheus Monitoring (Planned):** Metrics such as request latency, database query counts, and error rates will be exposed on the `/metrics` endpoint.
- **Grafana Dashboards:** Key metrics and alerting thresholds will be visualized in Grafana.

3.6.4 Deployment Targets

- The system can be deployed on cloud infrastructure (e.g., AWS EC2, DigitalOcean Droplets) or on-premise servers.
- Future plans include Kubernetes deployment using Helm charts with support for autoscaling.

- Secrets are managed through `.env` files in local/staging; production will use secure stores (e.g., AWS SSM, HashiCorp Vault).

These DevOps practices ensure that the platform remains reliable, secure, and rapidly deployable, supporting both development velocity and production-grade reliability.

3.7 REST API Overview

The backend system exposes a well-organized and comprehensive REST API to facilitate interaction between the frontend, mobile applications, and third-party clients. The API is modular, with routes grouped by business domains that reflect the underlying bounded contexts of the system. This modular routing ensures clarity, scalability, and maintainability of API endpoints.

Each API module adheres to REST principles:

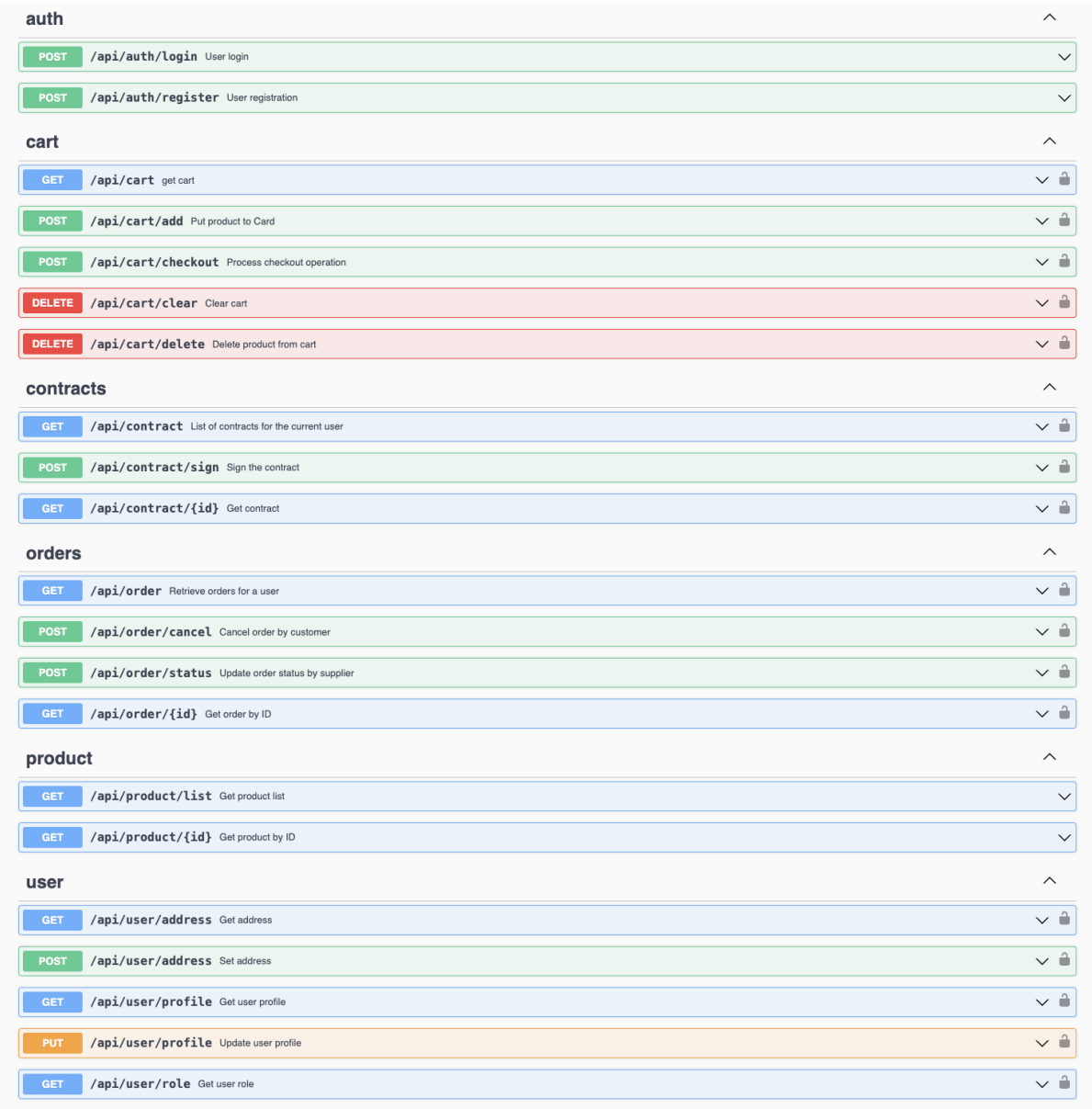
- **Resource-based paths:** URL structures correspond to nouns (e.g., `/cart`, `/order`, `/contract`) that represent business entities.
- **HTTP Verbs:** Methods like `GET`, `POST`, `PUT`, `DELETE` reflect standard CRUD operations.
- **Statelessness:** Each request contains all necessary information, primarily via JWT tokens in the Authorization header.
- **Consistent Naming:** Endpoints follow `snake_case` naming and pluralized resource paths where applicable.

3.7.1 API Documentation with Swagger

All routes are automatically documented using Swagger/OpenAPI, allowing developers to:

- Explore and test endpoints interactively.
- Understand expected parameters, request bodies, and response formats.
- Detect authentication requirements and error codes.

The Swagger UI is hosted at: `/swagger/index.html`



3.7.2 Sample Endpoints by Module

Auth Module

- `POST /api/auth/login` — Authenticate and return JWT token
- `POST /api/auth/register` — Create new user account

Cart Module

- `GET /api/cart` — View current user's cart
- `POST /api/cart/add` — Add product to cart
- `DELETE /api/cart/clear` — Clear all items in cart

Order Module

- `GET /api/order/{id}` — Get order details by ID
- `POST /api/order/cancel` — Cancel an order if still pending

Contract Module

- `GET /api/contract/{id}` — View a delivery contract
- `POST /api/contract/sign` — Sign contract as client or supplier

User Module

- `GET /api/user/profile` — Retrieve profile details
- `PATCH /api/user/profile` — Update name or phone number

Supplier Module

- `GET /api/supplier/orders` — View incoming orders
- `POST /api/supplier/accept` — Accept or reject an order
- `PUT /api/supplier/settings` — Update delivery and pricing rules

3.7.3 Authentication Headers

Most API routes require authentication via a Bearer token:

`Authorization: Bearer <JWT_TOKEN>`

Tokens are validated by middleware and used to inject user context into handlers.

3.7.4 Response Standards

All responses follow a consistent JSON structure:

```
{
  "status": "success",
  "data": { ... },
  "message": "Optional message"
}
```

Errors return:

```
{
  "status": "error",
  "message": "Invalid token"
}
```

This RESTful architecture ensures easy integration, minimal learning curve for developers, and high compatibility with frontend and mobile platforms.

3.7.5 Module Communication Example

This section demonstrates how different modules communicate internally using interface abstraction, particularly focusing on the interaction between the order and contract modules. The system follows a clean separation of concerns where modules depend on defined interfaces rather than concrete implementations, improving modularity, testability, and scalability.

Overview

- The order module calls the contract module when an order status transitions to `InProgress`.
- The contract module manages signing logic by both supplier and customer.
- Communication is encapsulated via the `IContractService` interface.

1. Contract Interface in `order/service.go`

```go
type IContractService interface {
        CreateContract(ctx context.Context, orderID, supplierID, customerID int64,
        ↪  content string) (int64, error)
}
```

This interface allows the order module to trigger contract creation without being tightly coupled to the contract module's implementation.

2. Triggering Contract Creation on Status Change

```go
if order.StatusID == model.Pending && newStatusID == model.InProgress {
        _, err := s.contractClient.CreateContract(
                ctx,
                order.ID,
                order.SupplierID,
                order.CustomerID,
                fmt.Sprintf("Contract for order #%d", order.ID),
        )
        if err != nil {
                return err
        }
}
```

This logic inside `UpdateOrderStatusBySupplier` method triggers contract creation automatically when an order enters the `InProgress` state.

3. Contract Creation Logic in `contract/service.go`

```go
func (s *Service) CreateContract(ctx context.Context, orderID, supplierID,
↪  customerID int64, content string) (int64, error) {
        contract := &model.Contract{
                OrderID:    orderID,
                SupplierID: supplierID,
                CustomerID: customerID,
                Content:    content,
                Status:     model.StatusCreated,
                CreatedAt:  time.Now(),
        }
        return s.repo.Create(ctx, contract)
}
```

This function constructs and persists a contract entity based on the order data and triggers downstream contract lifecycle handling.

4. Contract Signing by Either Party

```
func (s *Service) SignContract(ctx context.Context, contractID int64, role int,
↪  signature string) error {
        contract, err := s.repo.GetByID(ctx, contractID)
        if err != nil {
                return err
        }

        switch role {
        case orderModel.CustomerRole:
                if !contract.SupplierSig.Valid {
                        return fmt.Errorf("supplier has not signed the contract
                        ↪  yet")
                }
        case orderModel.SupplierRole:
                // proceed
        default:
                return fmt.Errorf("unknown role")
        }

        err = s.repo.SignByParty(ctx, contractID, role, signature)
        if err != nil {
                return err
        }

        if role == orderModel.CustomerRole && contract.SupplierSig.Valid ||
                role == orderModel.SupplierRole && contract.CustomerSig.Valid {
                return s.repo.MarkAsSigned(ctx, contractID)
        }

        return nil
}
```

This logic governs how the contract becomes fully signed once both parties (supplier and customer) have submitted their signatures.

5. Wiring the Contract Client in `order/service/service.go`

```
func NewService(
        repo IOrderRepository,
        supplierClient ISupplierClient,
        productClient IProductClient,
        contractClient IContractService,
        tx db.TxManager,
) *OrderService {
        return &OrderService{
                orderRepo:      repo,
                supplierClient: supplierClient,
                productClient:  productClient,
                contractClient: contractClient,
                txManager:      tx,
        }
}
```

This constructor function injects the `contractClient` dependency into the order service, enabling seamless inter-module calls while respecting modular boundaries.

4 Development of the Technical Solution

4.1 Web Interface Implementation

The web frontend of `Zhan.Store` is implemented using `Next.js` (App Router) with TypeScript and Tailwind CSS for styling. This modern stack ensures type safety, component reusability, and efficient build-time rendering with responsive UI support.

The project directory is structured to follow Next.js conventions with a clean separation of concerns. The most relevant parts of the structure are shown below:

```
src/
 app/
    page.tsx                // Home page
    catalog/page.tsx        // Product listing
    product/[id]/page.tsx   // Product details
    cart/page.tsx           // Shopping cart
    checkout/page.tsx       // Order placement
    orders/page.tsx         // Order history
    orders/[id]/page.tsx    // Order details and documents
    login/page.tsx          // Login page
    register/page.tsx       // Registration
    profile/page.tsx        // User profile
 components/                 // UI components (e.g., Navbar, Footer, ProductCard)
 context/                    // Global state management (e.g., AuthContext,
↪  CartContext)
 lib/axios.ts                // Axios config for backend API requests
 types/index.ts              // Shared TypeScript interfaces
```

All routing is handled via the App Router, which provides file-based routing and nested layouts. For example, the `/orders/[id]/page.tsx` file dynamically renders order details and the acceptance certificate.

UI Technologies

- Tailwind CSS for fast and utility-first styling
- Context API for global state management

- Axios for secure backend API communication using JWT tokens

The result is a responsive and accessible user interface that feels native on both desktop and mobile platforms.

Functional Highlights

- **Dynamic Cart Validation:** Validates minimum purchase amount per supplier
- **Order Segmentation:** Splits orders across suppliers with separate contracts
- **Document Rendering:** Users can view signed contracts and delivery acts

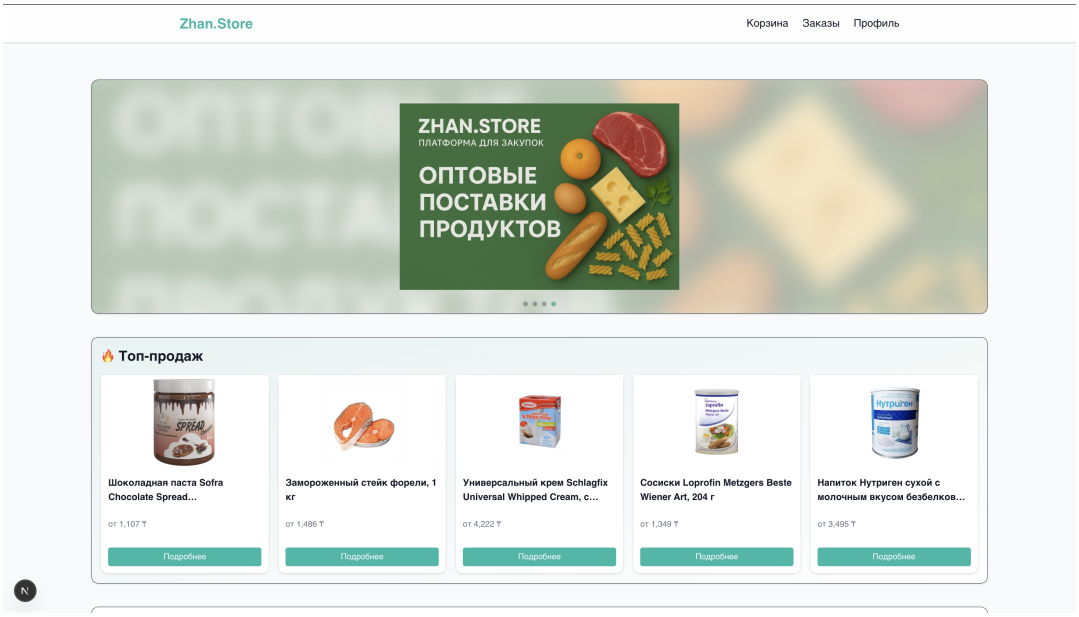


Рис. 4.1: Home Page UI

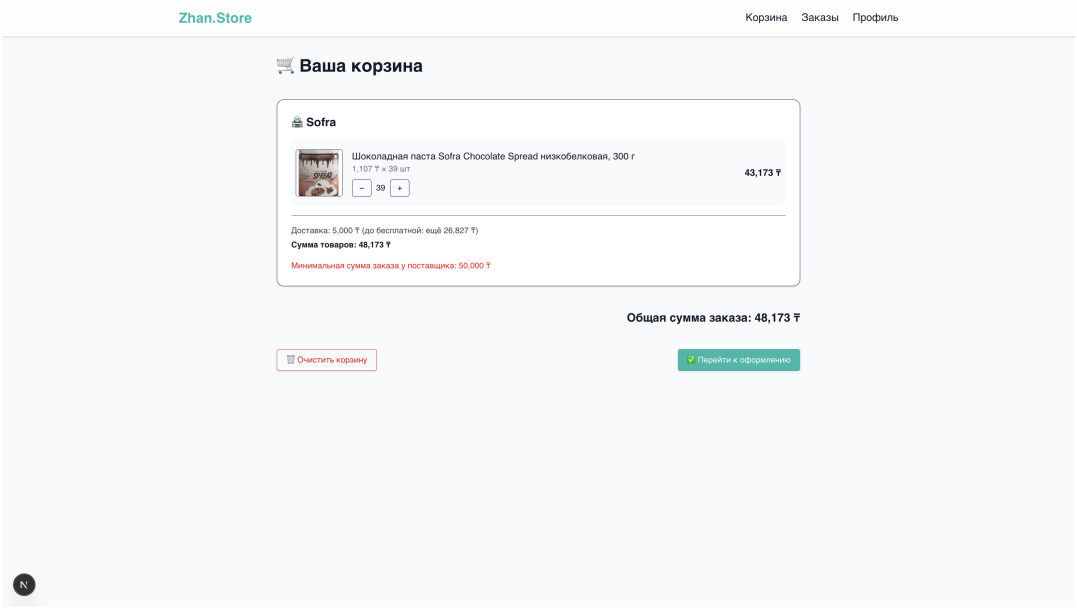


Рис. 4.2: Cart Page

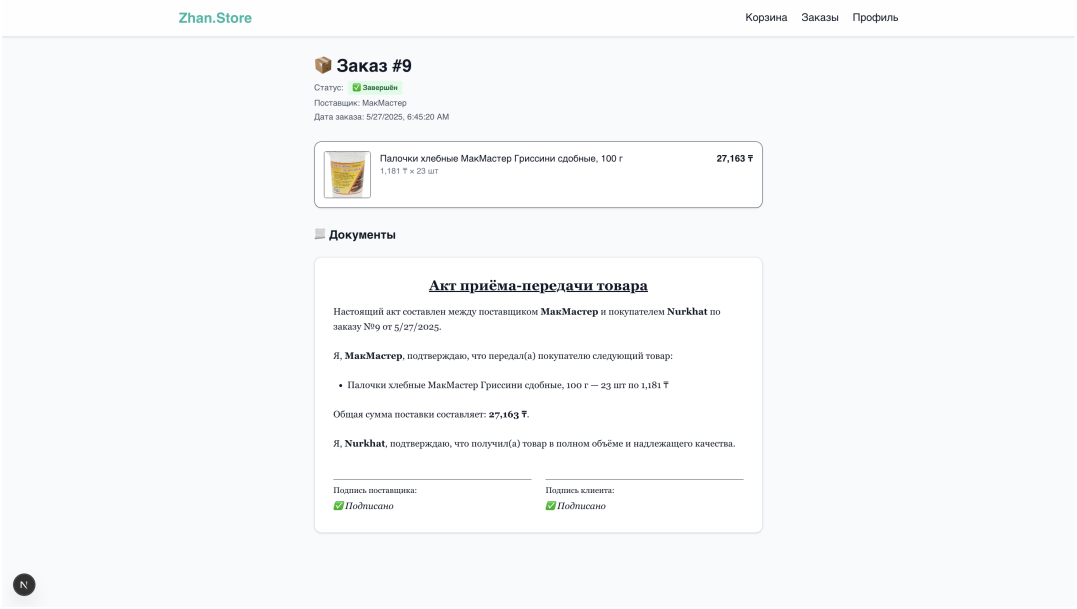


Рис. 4.3: Order Receipt and Document View

The web interface of `Zhan.Store` is designed with usability, clarity, and responsiveness in mind. Developed using modern front-end frameworks, the application enables businesses to browse, manage, and complete B2B purchases with ease.

4.1.1 Home Page, Catalog, Product Page

The landing page provides instant access to promotions and top-selling items. It features a responsive slider banner, category highlights, and a product showcase with clear pricing and supplier data.

Users can explore product categories and access detailed information on individual product pages, including minimum order requirements, supplier information, and pricing.

4.1.2 Cart and Checkout Flow

The cart page summarizes selected items grouped by supplier, calculating subtotal and delivery fees. A dynamic validation system checks whether the minimum order threshold for each supplier is met.

At checkout, the order is segmented by supplier, generating multiple purchase contracts behind the scenes. This design streamlines multi-vendor purchasing and simplifies fulfillment operations.

4.2 Mobile Application Features

The mobile application is developed using Flutter, enabling cross-platform deployment for both Android and iOS while sharing a single codebase. The architecture follows the MVVM (Model-View-ViewModel) pattern, enhanced by BLoC for reactive state management and Provider for dependency injection.

Authentication is a key entry point for users, implemented using a layered approach:
Authentication Module Overview

- **UI (Presentation Layer):** SignIn and SignUp screens using reusable components
- **State Management:** AuthBloc and AuthState control transitions
- **ViewModel Layer:** Transforms inputs into domain actions
- **Domain Layer:** Business logic abstraction
- **Data Layer:** AuthRepository connects to backend APIs

```
lib/
  core/services/auth_service.dart
  data/auth/models/login_model.dart
```

```
data/auth/repo/auth_repository.dart
domain/auth/auth_service.dart
presentation/auth/
    blocs/auth_bloc.dart
    pages/sign_in.dart
    view_model/sign_in_view_model.dart
```

Mobile UI Features

- Product catalog browsing with infinite scroll
- Responsive cart and supplier-specific pricing indicators
- Touch-optimized filtering and navigation

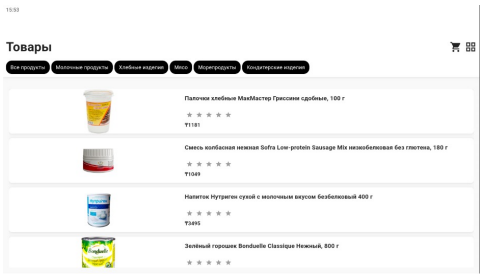


Рис. 4.4: Mobile Product Listing

The mobile version maintains all core functionalities of the web app. Users can browse products by category, manage the cart, place orders, and view contracts. The UI prioritizes readability and touch-friendly navigation for a seamless on-the-go experience.

4.3 Frontend-to-Backend Integration

The frontend connects to the backend API using `Axios`, a promise-based HTTP client for JavaScript. The configuration is centralized in the `src/lib/axios.ts` file, ensuring a consistent and secure setup for all API requests:

```
import axios from "axios";

const api = axios.create({
    baseURL: "http://188.227.35.6:8080/api",
    withCredentials: false,
});
```

```
7
8  api.interceptors.request.use((config) => {
9      const token = localStorage.getItem("access_token");
10     if (token) {
11         config.headers.Authorization = `Bearer ${token}`;
12     }
13     return config;
14 });
15
16 export default api;
```

This configuration achieves the following:

- Sets the default `baseURL` for all API calls to the backend.
- Automatically attaches the `Authorization` header with a JWT token retrieved from `localStorage`.
- Enables stateless, secure communication between frontend and backend.

The use of centralized configuration and request interceptors simplifies authentication logic across the application and ensures that protected endpoints remain secure without duplicating code in individual components.

4.4 Supplier Workflow, Contracts, and Payment Integration

The post-order process within `Zhan.Store` involves a tightly connected flow of contract generation, digital signing, supplier coordination, and payment. This ensures that both buyers and suppliers have a secure, traceable, and compliant transaction experience.

4.4.1 Order Lifecycle and Status Flow

Suppliers manage incoming orders via their personal dashboard. Orders progress through the following key stages:

- **Pending** – Order created and awaiting supplier review
- **In Progress** – Supplier has approved, and the contract is signed by both parties

- **Delivered** – Goods have been shipped, awaiting buyer confirmation
- **Completed** – Delivery has been accepted and confirmed

Each transition in status reflects a backend event, often involving API interactions, contract changes, or digital signatures.

4.4.2 Contract Generation and Digital Signatures

Once an order is placed and confirmed by the buyer, the backend invokes the `contractClient.CreateContr` method to create a formal contract record. This contract includes:

- Order ID
- Buyer and supplier identifiers
- Pricing and product descriptions
- Terms and delivery expectations

Contracts are rendered in PDF-like view on the frontend for both parties. Signing the contract is currently implemented as a checkbox-based confirmation, but future releases will integrate Kazakhstan's EDS (Electronic Digital Signature) system for legally binding validation.

4.4.3 Supplier Interaction with Contracts

Suppliers can:

- View pending orders and their attached contracts
- Approve or reject based on product availability or pricing
- Sign the contract once confirmed
- Track delivery and update fulfillment status

Backend modules such as `contract/service.go` manage signature status updates and store digital approval flags.

4.4.4 Payment Integration with Fondy

`Zhan.Store` uses the Fondy gateway to process secure online payments. This gateway supports:

- Credit and debit cards
- Bank transfers
- Digital wallets (Google Pay, etc.)

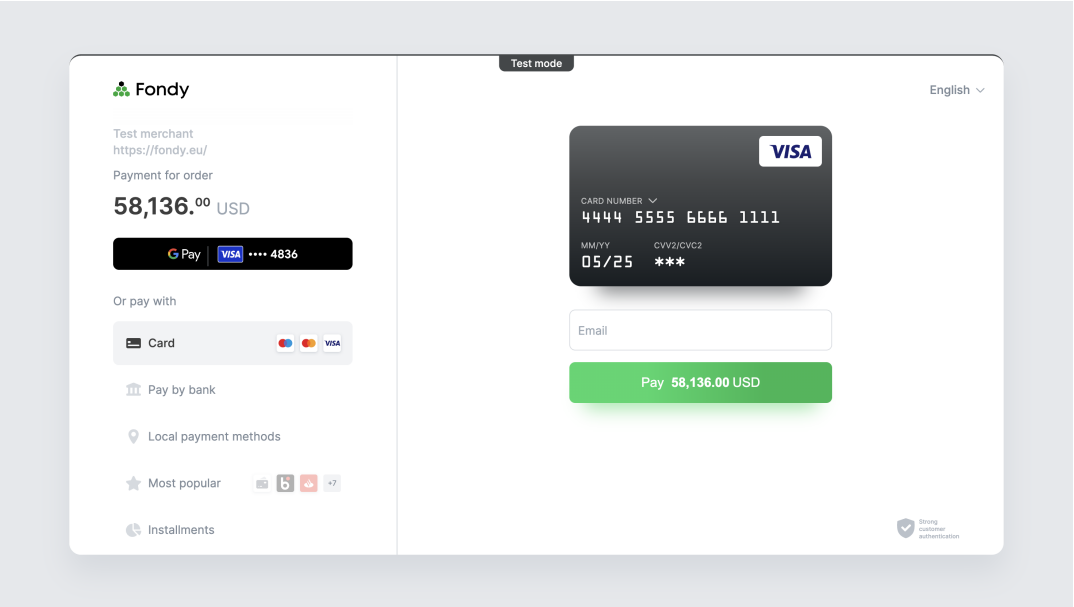


Рис. 4.5: Fondy Payment Gateway

The frontend initiates a payment redirect to Fondy's hosted interface using a dynamically generated `order_id`. Upon successful payment, the Fondy callback notifies the backend, which updates the order's status to `Paid` and allows contract signing.

4.4.5 Digital Signature Roadmap

Although initial releases use checkbox confirmations, future upgrades will support:

- Integration with Kazakhstan's National Certification Authority (NCA)
- Role-based access to private EDS keys
- Legally enforceable signing process and storage of signature artifacts

Conclusion

The development of the `Zhan.Store` platform represents a significant step toward the digital transformation of procurement and contract management for small and medium-sized enterprises (SMEs) in Kazakhstan. Through careful analysis, research, and full-stack implementation, the system addresses long-standing inefficiencies in traditional procurement methods, such as fragmented communication, manual documentation, and lack of transparency.

The platform introduces a comprehensive and localized digital solution tailored to the unique business, legal, and infrastructural realities of Kazakhstan. It integrates a modular monolithic backend in Golang, a modern web interface using Next.js with TypeScript and Tailwind CSS, and a cross-platform mobile application developed in Flutter. These components work together seamlessly via secure REST APIs, ensuring consistency, scalability, and maintainability.

Key innovations include:

- Automated order segmentation and contract generation per supplier;
- A full digital workflow from catalog browsing to contract signing;
- Integration with local payment systems such as Fondy;

This solution not only reduces procurement time and human error but also empowers SMEs with better supplier visibility, compliance, and operational analytics. Moreover, the adoption of clean architecture, modular design, and DevOps practices ensures that the system is future-proof and ready for further enhancements, including microservices scaling, AI-assisted supplier ranking, and blockchain-based transparency.

In conclusion, `Zhan.Store` is not just a technological product—it is a catalyst for economic modernization. By equipping SMEs with the tools necessary to streamline operations and engage confidently in digital commerce, it contributes meaningfully to Kazakhstan's broader goals of digitalization, regional development, and business innovation.

References

[1] Ramina Nasyrova and Magzhan Saidalin.
E-commerce market in kazakhstan – flagship of central asia.
https://bakertilly-ca.com/en/insights/kazahstanskij-rynok-e-com-flagman-v-czentralnoj-azii/, 2023.

[2] Mirzo Iskandar Gulamov and Yerlan Khassenbekov.
Constraints to sme growth in kazakhstan and how to overcome them.
https://development.asia/insight/constraints-sme-growth-kazakhstan-and-how-overcome-them, 2020.

[3] Pavel Mitrofanov and Arseniy Klaz.
Electronic trading platforms market: Digital transformation continues.
https://raexpert.ru/researches/etpb/etp_market_2024/, 2024.

[4] K. A. Belokrylov and N. N. Visitskiy.
Impact of digitalization on the accessibility of public procurement for small businesses.
https://cyberleninka.ru/article/n/vozdeystvie-tsifrovizatsii-na-dostupnost-gosudarstvennyh-zakupok-dlya-malogo-biznesa, 2022.

[5] Organisation for Economic Co-operation and Development.
Public procurement in kazakhstan: Reforming for efficiency.
https://www.oecd.org/ru/publications/8c22ccf8-ru.html, 2019.

[6] A. A. Ivanov and B. B. Petrov.
Challenges of implementing automation in core processes of small and medium-sized manufacturing enterprises.
https://cyberleninka.ru/article/n/problemy-vnedreniya-avtomatizatsii-v-osnovnye-protsessy-proizvodstvennyh-predpriyatiy-malogo-i-srednego-biznesa, 2021.

[7] Silvia Moreira, Henrique S. Mamede, and Arnaldo Santos.
Business process automation in smes.

https://www.researchgate.net/publication/370145022_Business_Process_Automation_in_SMEs, 2023.

[8] Jahedul Alam Bhuiyan, Bebe Asma, and Sabuj Chandra Bhowmik.
Digital procurement practices in smes: Comparative cases of advanced and emerging economies.
https://www.researchgate.net/publication/380429779_Digital_procurement_practices_in_SMES_comparative_cases_of_advanced_and_emerging_economies, 2024.

[9] World Bank.
Assessment of the public procurement systems of the republic of kazakhstan.
https://openknowledge.worldbank.org/entities/publication/1316893e-dff1-5940-8879-bcd11a08f1f6, 2023.

[10] J. Bamini and Sandhya A.
A systematic review on e-procurement by smes.
https://www.researchgate.net/publication/365824311_A_SYSTEMATIC_REVIEW_ON_E-PROCUREMENT_BY_SMEs, 2022.

[11] Ermos Michael Jama, Bupe Getrude Mwanza, and Erastus Mishengu Mwanaumo.
E-procurement adoption barriers encountered by small and medium-sized enterprises (smes) in the republic of south sudan.
https://ijcsacademia.com/index.php/journal/article/view/49, 2024.

[12] Molebaleso Lydia Ntshingila.
Procurement performance improvement through adoption of e-procurement: A study of small and medium enterprises.
https://journal.eu-jr.eu/social/article/download/3703/2666/, 2025.

[13] Vitaliy Ten.
Comparative analysis of public procurement methods of the republic of kazakhstan and other countries.
https://hrcak.srce.hr/file/460579, 2024.

[14] Ilyas Masudin, Ganis Dwi Aprilia, Adhi Nugraha, and Dian Palupi Restuputri.
Impact of e-procurement adoption on company performance: Evidence from indonesian manufacturing industry.
https://doi.org/10.3390/logistics5010016, 2021.

[15] Chinwe Chinazo Okoye, Daniel Ogochukwu Nwankwo, Njideka Maryann Okeke, Ekene Ezinwa Nwankwo, and Solomon Uchechukwu Eze.
Electronic commerce and sustainability of smes in anambra state.
https://doi.org/10.26480/mecj.01.2023.32.41, 2023.

[16] Kai-Kit Soong, Elsadig Musa Ahmed, and Khong Sin Tan.
Factors influencing malaysian small and medium enterprises adoption of electronic government procurement.
https://doi.org/10.1108/jopp-09-2019-0066, 2020.

[17] Ahmad Marei.
The effect of e-procurement on financial performance: Moderating the role of competitive pressure.
https://doi.org/10.5267/j.uscm.2022.3.009, 2022.

[18] Ziad Hmwd A. Almtiri, Shah Jahan Miah, and Nasimul Noman.
Application of e-commerce technologies in accelerating the success of sme operation.
https://doi.org/10.1007/978-981-19-1610-6_40, 2022.

[19] Omar Hasan Salah and Mohannad Moufeed Ayyash.
E-commerce adoption by smes and its effect on marketing performance: An extended of toe framework with ai integration, innovation culture, and customer tech-savviness.
https://doi.org/10.1016/j.joitmc.2023.100183, 2023.

[20] Wai Peng Wong, M. C. Sinnandavar, and S. O. Soh.
The effect of digital procurement and supply chain innovation on smes performance.
https://doi.org/10.5267/j.ijdns.2022.4.015, 2022.

[21] Jerimiah Madzimure, Chengedzai Mafini, and Manillal Dhurup.
E-procurement, supplier integration and supply chain performance in small and medium enterprises in south africa.
https://doi.org/10.4102/sajbm.v51i1.1838, 2020.

Appendix A: Code Listings

This appendix contains code listings related to the system architecture and backend/frontend implementation.

1. Extracting JWT Claims from Context (Auth Middleware)

```
claims, ok := c.Request.Context().Value(contextkeys.UserKey).(*jwt.Claims)
if !ok || claims == nil {
        c.JSON(http.StatusUnauthorized, modelApi.ErrorResponse{Err:
        ↪  modelApi.ErrUnauthorized.Error()})
        return
}
```

Used in all protected handlers to verify user authentication.

2. Role-Based Access Control (RBAC)

```
if claims.Role != model.SupplierRole {
        c.JSON(http.StatusForbidden, modelApi.ErrorResponse{Err: "only suppliers can
        ↪  perform this action"})
        return
}
```

Simple role check restricting access to suppliers only.

3. Starting a Transaction with Read Committed Isolation

```
err := s.txManager.ReadCommitted(ctx, func(ctx context.Context) error {
        // business logic inside transaction
        return nil
})
```

Ensures safe data modifications with nested transaction support.

4. SQL Construction with Squirrel and Logging

```
builder := sq.
        Update("orders").
        Set("status_id", newStatus).
        Where(sq.Eq{"id": orderID}).
        PlaceholderFormat(sq.Dollar)

query, args, _ := builder.ToSql()
_, err := r.db.DB().ExecContext(ctx, db.Query{Name: "order.update_status", QueryRaw:
↪  query}, args...)
```

Safe and readable SQL building without string concatenation.

5. Entity Conversion Between Layers (Converter Pattern)

```
func ConvertOrderToAPI(order *serviceModel.Order) *apiModel.Order {
        return &apiModel.Order{
                ID:        order.ID,
                Status:    convertStatusIDToString(order.StatusID),
                OrderDate: order.OrderDate.Format("2006-01-02T15:04:05Z07:00"),
                Supplier: &apiModel.Supplier{
                        ID:   order.Supplier.ID,
                        Name: order.Supplier.Name,
                },
                ProductList: convertOrderProductsToAPI(order.ProductList),
        }
}
```

Maintains clean API and service layer separation.

6. Centralized Error Definitions

```
var (
        ErrUnauthorized = errors.New("api: unauthorized")
        ErrNoRows       = errors.New("models: no rows")
)
```

Facilitates reusable and consistent error handling.

7. Filtering Orders by User Role

```
switch role {
case model.CustomerRole:
        orders, errTx = s.orderRepo.OrdersByUserID(ctx, userID)
case model.SupplierRole:
        orders, errTx = s.orderRepo.OrdersBySupplierID(ctx, userID)
}
```

Delegates order queries based on user role.

8. Password Hashing with bcrypt

```
func HashPassword(password string) (string, error) {
        bytes, err := bcrypt.GenerateFromPassword([]byte(password),
        ↪  bcrypt.DefaultCost)
        if err != nil {
                return "", err
        }
        return string(bytes), nil
}
```

Ensures secure password storage during registration.

9. Parsing Query Parameters for Pagination

```
limit := c.DefaultQuery("limit", "20")
offset := c.DefaultQuery("offset", "0")

limitInt, _ := strconv.Atoi(limit)
offsetInt, _ := strconv.Atoi(offset)

input := modelApi.ProductListInput{
        Limit:  limitInt,
        Offset: offsetInt,
}
```

Universal method for parsing pagination parameters.

Appendix B: Web-Platform Interface

Below are screenshots demonstrating the user interface and workflow of the Zhan.Store web platform prototype.

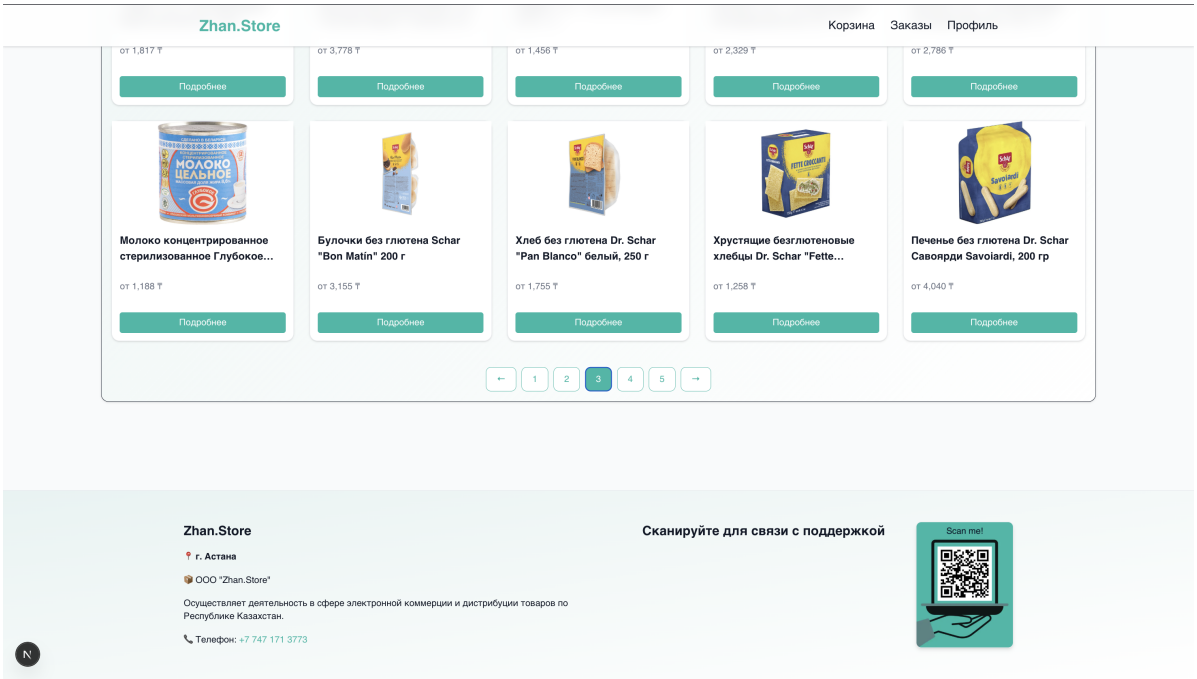


Рис. 1: Product listings with pagination

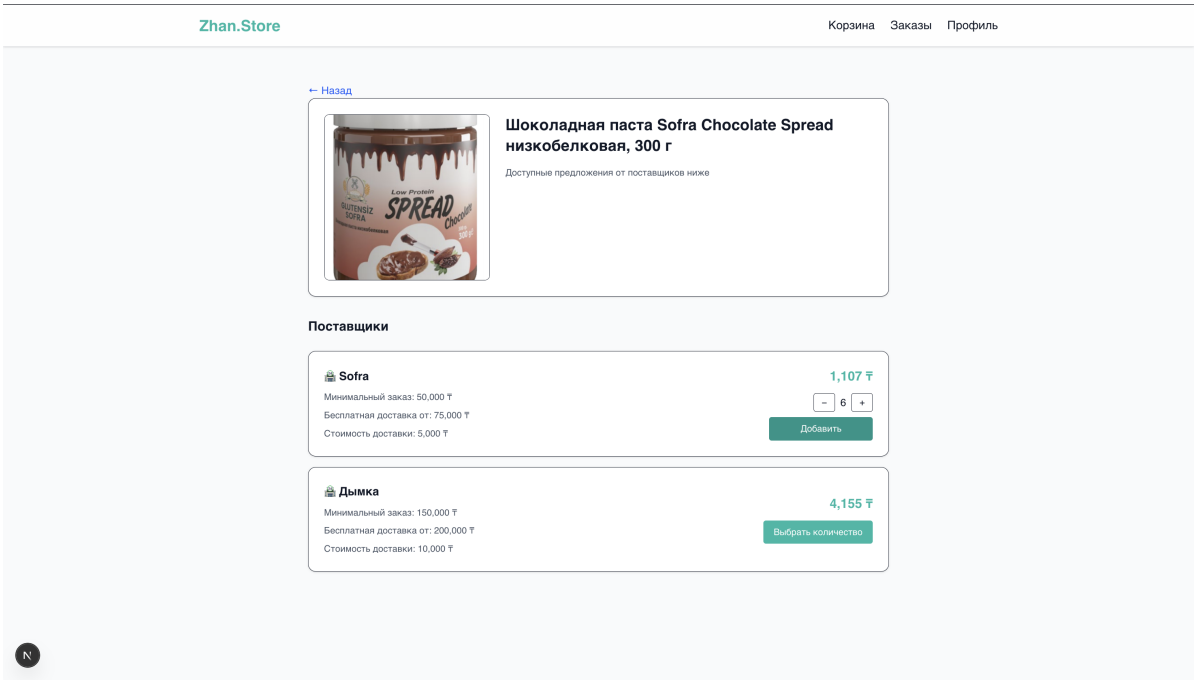


Рис. 2: Product detail page with supplier comparison

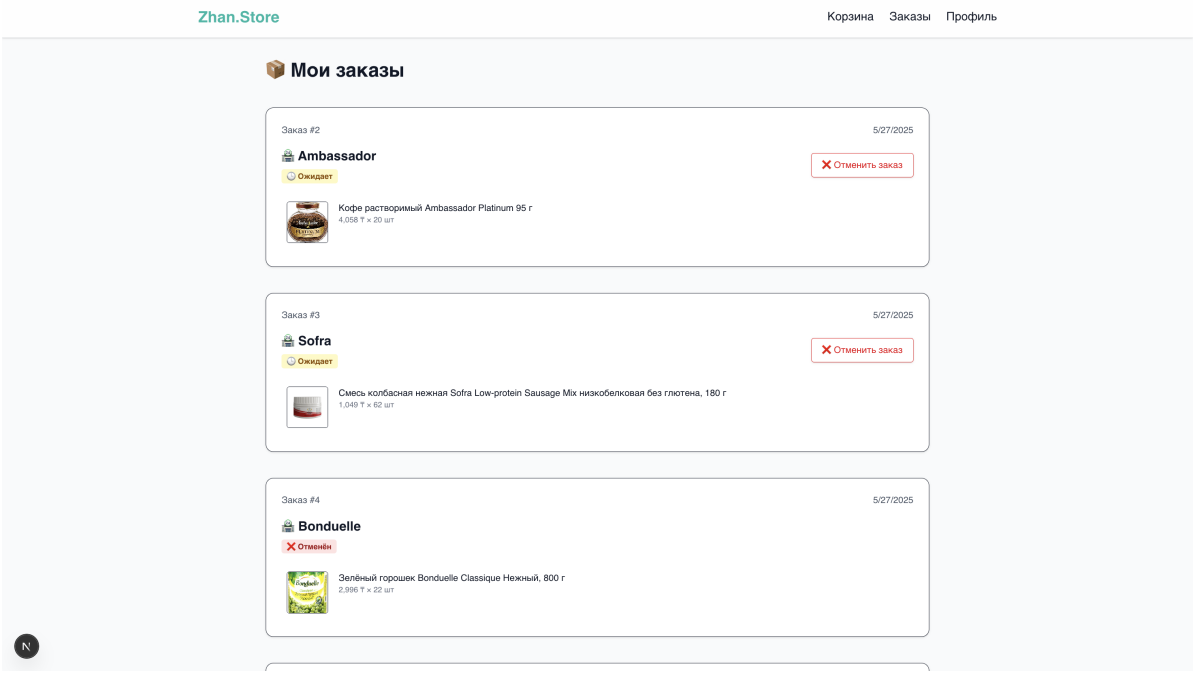


Рис. 3: Orders dashboard with status indicators

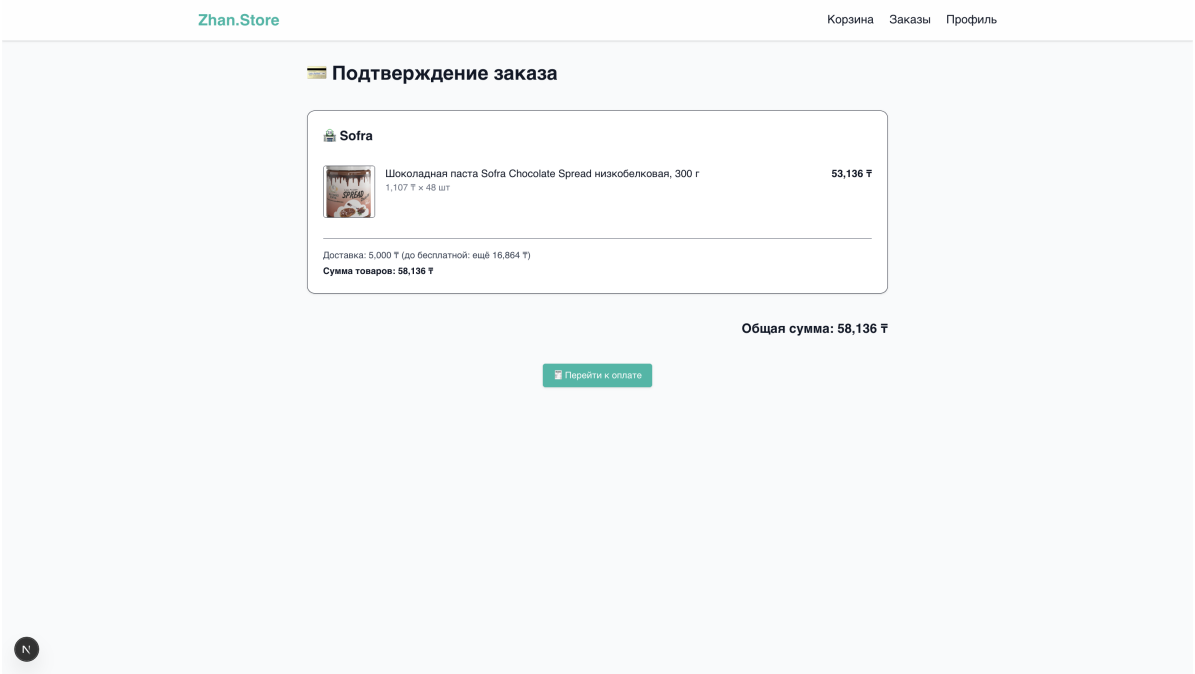


Рис. 4: Order confirmation screen before payment

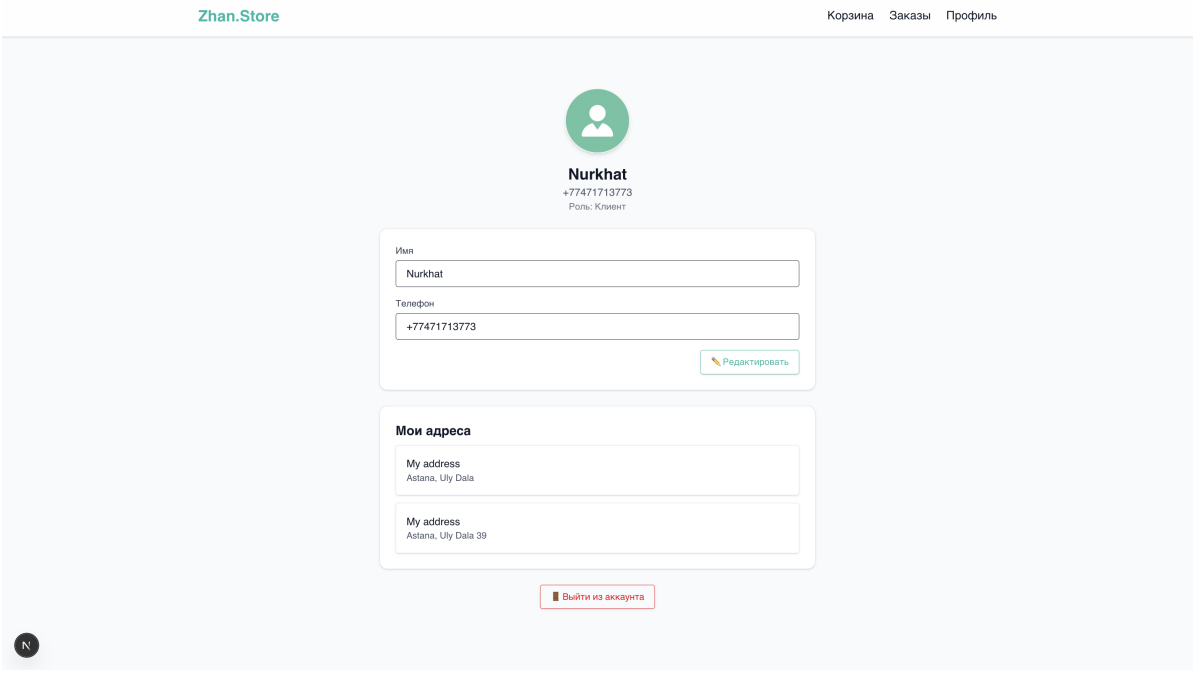


Рис. 5: User profile and address management

Appendix C: Mobile Platform Interface

This appendix includes screenshots of the mobile version of the platform, displaying key functionality such as product browsing, filtering, supplier selection, and cart interaction.

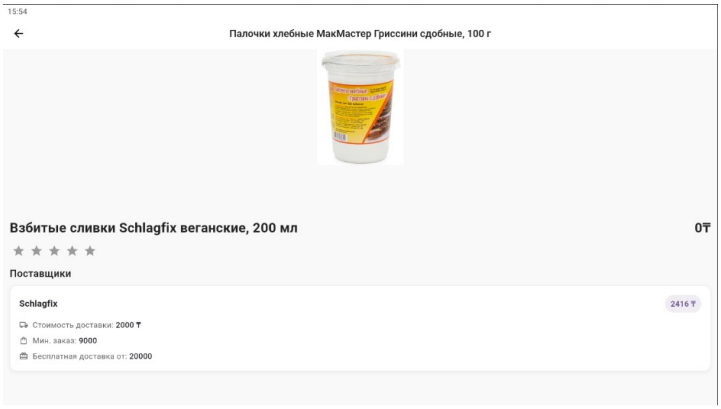


Рис. 6: Mobile product detail with supplier and pricing

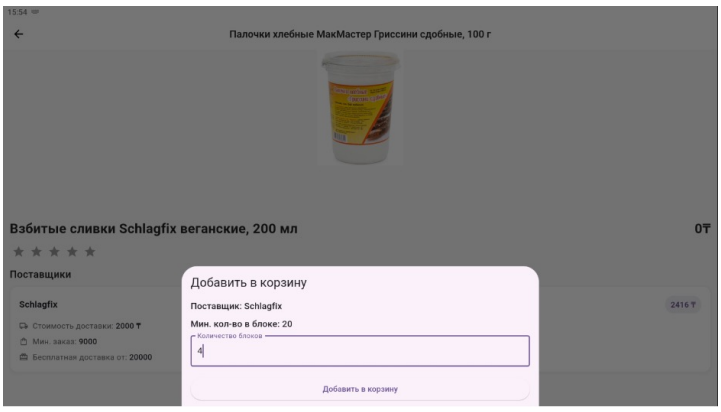


Рис. 7: Block-based quantity entry modal for supplier